

14

Asking a KG questions with natural language

This chapter covers

- Understanding the limitations of RAG in complex scenarios
- Building an advanced question-answering system that mimics domain expertise on KGs
- Transforming query results into meaningful, actionable summaries

In this chapter, we will explore how to build an advanced system that can answer questions effectively. Using a law enforcement example as our guide, we'll compare the retrieval-augmented generation (RAG) approach and our new “expert emulation” method for capturing the expertise of skilled information retrieval.

The framework we'll develop rests on several pillars:

- Understanding and properly routing different types of user questions
- Extracting and representing domain knowledge in a form that LLMs can use effectively
- Implementing expert-like reasoning patterns for query construction

- Ensuring that results are presented in meaningful, actionable ways

This framework is designed to integrate with a front-end layer, ensuring that the question-answering system can be presented to end users through a graphical interface. This integration-first approach influences many of our design decisions throughout the chapter, from how we structure query responses to how we handle data visualization.

This chapter walks through each of these components, using examples to illustrate the key concepts and design decisions; chapter 15 will provide a hands-on implementation guide, complete with an end-to-end example in a practical setting. By the end of these two chapters, you'll have both a deep understanding of the underlying concepts and the knowledge needed to implement an expert-emulated question-answering system for your own knowledge graphs.

Throughout this chapter, we'll use examples from law enforcement, but the principles and patterns we discuss can be adapted to any domain where expert knowledge needs to be systematically applied to complex data structures. Whether you're working in healthcare, finance, or scientific research, the framework will provide a solid foundation for building intelligent question-answering systems that use domain expertise.

14.1 Querying a knowledge graph in the policing domain

Imagine that you are an experienced analyst in a large law enforcement agency. Every day, you face the challenge of making sense of diverse and often overwhelming data streams. The policing domain is inherently complex and dynamic, with a mission to investigate crimes, prevent criminal activities, and protect the community. Your work sits at the heart of this mission.

The policing ecosystem encompasses a wide range of roles and responsibilities. Frontline officers respond to incidents and collect on-the-ground information, detectives and investigators connect the dots to solve cases, forensic experts analyze evidence, and analysts like you uncover patterns and trends from disparate data sources. Each role contributes vital pieces to the puzzle, and their collective efforts drive the interconnected processes in law enforcement.

A simple crime report can set off a cascade of activities: interviews, evidence collection, database searches, and more. Each step in this chain generates and consumes data, making it vital to integrate and organize this information effectively. Recognizing the need for a unified approach, your agency has invested in building a KG: a single source of truth that structures and connects roles, processes, and data. In this section, we will concentrate on extracting information, uncovering knowledge, and deriving insights from a fully constructed, ready-to-query KG.

14.1.1 Enabling domain experts with knowledge graphs

Seasoned analysts possess sharp intuition, a deep contextual understanding, and the ability to discern patterns in complex scenarios. In policing, these qualities often

outweigh technical expertise when it comes to interpreting data and making impactful decisions.

Despite this, querying KGs is traditionally seen as a task requiring deep technical skills—skills that are outside the typical toolkit of domain experts. However, empowering analysts like you to interact directly with KGs opens immense possibilities:

- *Using expertise*—You, as a domain expert, bring unparalleled contextual knowledge to the table. This enables you to frame nuanced and insightful queries, driving highly accurate and relevant data retrieval.
- *Timely decisions*—Direct access to KGs eliminates bottlenecks, allowing you to make quicker decisions: a necessity in critical fields like law enforcement, where time is often of the essence.
- *Overcoming technical barriers*—Most domain experts, although masters of their fields, lack the technical training to write complex queries for KGs. Removing this barrier expands the accessibility of these tools.
- *Resource efficiency*—By reducing reliance on IT teams or data scientists to generate queries, you free up technical resources for other priorities, making workflows more efficient and collaborative.
- *Skill specialization*—Your focus is on mastery of your domain, not on becoming a technical expert. Enabling intuitive KG interaction ensures that your energy remains devoted to what you do best.
- *New perspectives*—When you can engage directly with data, your unique perspective allows for the discovery of innovative insights and applications that technical users might overlook.
- *Enhanced problem-solving*—With the ability to explore data hands-on, you can uncover creative approaches to solving problems, tailored to your domain's needs.
- *Maximizing ROI*—KGs represent a significant investment. Ensuring their full use by empowering analysts maximizes their value and impact.
- *Cross-functional collaboration*—Easy access to KGs fosters collaboration across teams, bridging gaps between technical and nontechnical stakeholders by creating a shared platform for data access.
- *Shared understanding*—By enabling broader access to KGs, organizations promote a unified approach to data analysis and problem-solving, building alignment across roles and disciplines.

By breaking down these barriers, the potential of KGs can be unlocked, bringing insights and practical solutions to the forefront.

14.2 RAG for KG querying: Capabilities and challenges

It's important to remember that LLMs generate answers based on the data collected during their training phase. Although LLMs can answer questions that require specialized expertise, their utility diminishes for users who are already experts in their field.

Consider the types of questions a law enforcement official might ask an LLM. In this context, it's hard to think of questions beyond basic inquiries like, "What is the legal definition of probable cause?" or "How should an officer properly collect and preserve evidence at a crime scene?" The responses that an LLM can generate are likely to be of little use to trained professionals.

To be beneficial, an LLM should provide access to information that personnel can use to expedite their processes, whether those involve suspect identification, organized crime network analysis, or other complex tasks. The most popular approach to answering questions grounded in the data available to an organization, such as a law enforcement agency, is RAG, as we discussed in chapter 13.

Yet RAG faces critical limitations when retrieval is incomplete. To illustrate this, we'll examine how removing a single piece of context from a working example leads to opposite conclusions.

14.2.1 RAG effectiveness with complete context

Let's start with RAG working effectively when all relevant context is retrieved. Consider a request like "Generate a summary of witness statements related to CASE123." There is little chance that an LLM alone can effectively handle this request because the model has no prior knowledge of that specific case. Without any contextual information, the LLM cannot access the details necessary to provide a meaningful response. However, if we integrate that question with a list of all witnesses connected to CASE123, the situation changes significantly. It transforms into a straightforward summarization task, and LLMs excel at this type of work. When questions are accompanied by relevant contextual content, LLMs can generate richer and more pertinent answers.

To demonstrate this concept, Listing 14.1 shows a sample set of witness statements that would typically be retrieved from a case database and provided as context to the LLM.

Listing 14.1 Sample witness statements for LLM prompt context

1. Witness A, a barista at a local coffee shop, says: "The person who came in that day was wearing a unique jacket with a distinctive logo on the left sleeve. They ordered a latte and when they went to pay, they used their right hand to take out their wallet. I noticed that their left hand had a small bandage on the index finger."
2. Witness B, a neighbor who saw the perpetrator leaving the scene, says: "I didn't get a good look at their face, but I did notice that they were carrying a bag in their right hand. They seemed to be in a hurry and were walking quickly."
3. Witness C, a security guard at a nearby building, says: "I reviewed the security footage and I noticed that the person of interest was holding a phone up to their ear with their left hand while they were walking. They

seemed to be talking to someone and were carrying a bag in their right hand."

4. Witness D, a friend of the victim, says: "When I went to visit my friend, I noticed that some of their belongings had been moved around. I remember that a book on their bookshelf was out of place, but I can't recall anything unusual about it."
5. Witness E, a police officer who responded to the scene, says: "We found a footprint at the scene that we believe belongs to the perpetrator. The print shows that the person was wearing a shoe with a distinctive tread pattern on the sole, but we cannot determine which foot made the print."

Using these witness statements, let's build a prompt to test this idea. We'll use a reusable template, and the portions in curly brackets will be replaced with the relevant witness statement content and the question itself.

 You are a detective assistant helping answer the detective's question
 [Witness statements]
 {{ witness_statements }}
 [Question]
 {{ question }}

If we use the prompt to ask the LLM to "summarize the witness statements," the model provides these results:

 Here's a summary of the witness statements:

****Physical Characteristics:****

- Unique jacket with a distinctive logo on the left sleeve (Witness A)
- Small bandage on the left index finger (Witness A)

****Actions and Behaviors:****

- Used right hand to take out wallet (Witness A)
- Carrying a bag in right hand (Witnesses B and C)
- Held phone up to ear with left hand while walking (Witness C)
- Seemed to be in a hurry and walking quickly (Witness B)

****Scene Evidence:****

- Footprint with distinctive tread pattern, but unclear which foot made the print (Witness E)
- Book on bookshelf was out of place, but no other unusual details (Witness D)

The model understood the task and delivered a concise and relevant summary of the key details from the witness statements.

Now, let's ask whether the witness statements include anything related to a lead we are following. For example, if we use the prompt to ask "Is the perpetrator left-handed?"

the model's gives the following response. The output provides a detailed analysis of the relevant witness details, concluding that there is some evidence suggesting the suspect may be left-handed.



Let's analyze the witness statements to determine if the perpetrator is left-handed.

Witness A mentions that the perpetrator used their right hand to take out their wallet, which suggests that they might be right-handed. However, Witness A also notes that the perpetrator had a small bandage on the index finger of their left hand, which could indicate an injury. This doesn't necessarily imply that the perpetrator is left-handed, but it's worth considering.

Witness C says that the perpetrator was holding a phone up to their ear with their left hand while walking, which suggests that they might be left-handed. This is strong evidence pointing towards the perpetrator being left-handed.

Witness B and Witness E's statements don't provide any direct information about the perpetrator's handedness. Witness D's statement is also not relevant to this question.

Based on the available evidence, I would say that it's likely that the perpetrator is left-handed. The fact that they used their left hand to hold the phone to their ear in Witness C's statement is a strong indication of this. However, it's important to keep in mind that people can be ambidextrous or have varying degrees of handedness, so we shouldn't make any definitive conclusions just yet. Further investigation is needed to confirm our suspicions.

In this case, the LLM generated an answer that is consistent with the content we provided—the witness statements—and directed us to relevant pieces of information. This capability allows the LLM to assist in uncovering insights that may be critical to the investigation.

Now suppose the case in question is a murder investigation, and we want to find any connections between the victim and known criminal organizations. Given that people may be reluctant to discuss local criminal activities openly, the witness statements may not be useful. To obtain meaningful answers, we need more contextual information, and this is where LLMs can be particularly effective.

Additional context may include the victim's criminal record, prior convictions, personal history, associations or relationships, social media profiles, or any officer reports mentioning the victim or their family. This content must be retrieved from the respective sources, filtered for relevance, and then provided as context along with our question to the LLM. This retrieval, filtering, and contextual augmentation is what RAG aims to accomplish.

14.2.2 RAG fragility with incomplete retrieval

RAG is a highly effective approach with a wide range of applications. However, it comes with its own set of challenges. The success of RAG is closely tied to the effectiveness of the retrieval process.

LLMs cannot generate accurate answers if they are not provided with sufficient context; therefore, for RAG to produce a meaningful response, the retrieval process must be capable of locating the correct information.

Now let's see what happens with the same question but incomplete retrieval. Suppose we did not have a statement from witness C. We again ask the model "Is the perpetrator left-handed?" The following listing shows the response. Compare it to the previous response, and you can see how strongly it depends on the context provided: the absence of a single witness statement is enough to lead the model to the opposite conclusion!



```
Based on the witness statements, I think we can make an educated inference about the perpetrator's handedness.
```

```
Witness A, the barista, mentions that when the person went to pay, they used their right hand to take out their wallet. This suggests that the perpetrator is likely right-handed.
```

```
Additionally, Witness B, the neighbor, states that the perpetrator was carrying a bag in their right hand when they were leaving the scene, which further supports the idea that the perpetrator is right-handed.
```

```
There is no concrete evidence in the statements to suggest that the perpetrator is left-handed. The fact that Witness A mentions a bandage on the index finger of the perpetrator's left hand could be unrelated to their handedness.
```

```
So, based on the available information, it seems more likely that the perpetrator is right-handed rather than left-handed.
```

Figure 14.1 illustrates this process. Even though the AI agent confidently processes the available information, the incomplete context—caused by the retriever failing to identify critical documents—leads to wrong conclusions. This visualization emphasizes a key limitation of RAG systems: their outputs are only as good as their retrieval step, regardless of the model's confidence in its response.

In addition, RAG performs better when our sources of information can be divided into "passages" that contain fine-grained, independent pieces of information—essentially, many small blocks of information that can be used to construct a highly relevant context. But in practice, answers often span multiple documents, and the retrieval process may not capture all of them. Furthermore, some documents may be relevant even if they do not contain the specific answer we are searching for, which adds a layer of complexity to the task for the LLM.

When the retrieved passages lack the appropriate level of granularity, or if the retriever fails to accurately assess relevance, the context provided to the LLM becomes fragmented. This fragmented context can push the model toward incorrect assumptions, often leading it to generate plausible but inaccurate details. To address these challenges, we need an approach that captures the nuanced insight an expert would apply when querying a KG, combining structured retrieval with context-aware reasoning.

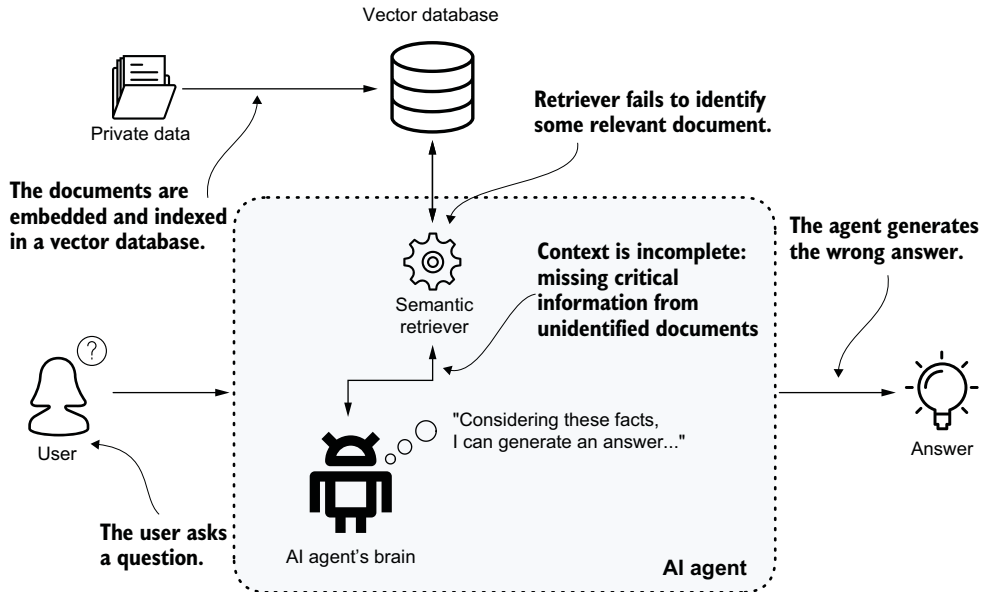


Figure 14.1 RAG system limitations. A RAG system can produce incorrect answers when the retriever fails to identify critical documents.

14.3 Schema-based approach for querying KGs

We have seen that RAG can sift through millions of documents to provide a pertinent and rich answer, provided the retriever delivers a comprehensive contextual picture to the LLM. Let's take a moment to scale this problem down to a more manageable size, say a dozen documents. Imagine that we ask a group of experts a question, but constrain them to apply a "human-powered" RAG-like approach. First they review the documents, marking any paragraphs that contain similar content or pertain to the same domain. From these, they select a dozen paragraphs that are the most relevant, which serve as a context. Once they have their context, they set aside everything else they have read and attempt to answer the question using only the selected pieces of information and their common sense, making assumptions if necessary.

This process is not impressive. Even though these are domain experts, when they're forced to work within RAG-like constraints, their approach lacks the depth and rigor that their genuine expertise would normally demand.

What would an expert do in this scenario? Imagine that we have a KG, and we need to answer a specific question. The first step is to understand the schema, which acts as the blueprint of the graph. For example, if the question involves people, it will likely involve a node labeled *Person*.

Understanding the schema is important because it allows us to convert specific requests like "show me the red Camaros seen in this area at that time" into a precise

traversal. This traversal might involve entities and relationships such as “Person—Owns_car—Car—Captured_by—ANPR (automatic number plate recognition).

The traversal can then be refined by applying constraints, such as requiring the car to be red, the model to be Camaro, and the ANPR camera to be located in a specific area and time frame. Deciding which nodes and relationships to select, as well as how to apply these constraints, is guided by the schema. Understanding the schema allows experts to write formal queries that pinpoint the specific information they are seeking, even amid vast amounts of data.

14.3.1 Understanding and using graph schemas

Understanding a graph schema—the data modeler’s structured interpretation of the domain—is like understanding the layout of a city before navigating through it: it lets us identify where specific types of information are stored and how they are linked. Figure 14.2 shows an example of the mental process an expert would follow when translating a natural-language question into a formal Cypher query. This translation process demonstrates how a solid understanding of the schema—knowing which nodes exist, their properties, and how they’re connected—is fundamental to constructing effective graph queries.

When we receive a query like “show me the red Camaros seen in this area at that time,” the initial step is to map the concepts mentioned in the query (e.g., red Camaros, area, time) to their corresponding nodes and relationships in the graph schema. This involves identifying the following:

- *Entities*—For instance, “red Camaro” maps to node labeled *Car* with properties such as *Color* and *Model*.
- *Relationships*—The query implies relationships like “*Person* owns *Car*” and “*Car* captured by *ANPRCamera*.”
- *Constraints*—These are the specific conditions that refine the traversal, such as the color being red, the model being Camaro, and the ANPR camera being located in a specified area.

This results in *constrained traversals*, which address questions like, “Among all these entities (cars, people, cameras), connected in this specific way (e.g., *OWNS_CAR*, *CAR_CAPTURED_BY_ANPR*), which ones possess the characteristics I’m interested in?”

Constrained traversals are the fundamental building blocks for constructing formal queries of any complexity. These traversals can be composed in various ways to build sophisticated queries that navigate intricate data relationships.

For example, we might start with an initial constrained traversal—possibly with aggregations—to identify specific nodes or subgraphs, which can then serve as starting points for further exploration in subsequent stages. This compositional approach allows us to address complex questions that require multiple layers of exploration and refinement. Ultimately, constrained traversals enable the translation of natural

language intent into structured queries that a graph database can execute, effectively bridging the gap between human language and machine-readable data.

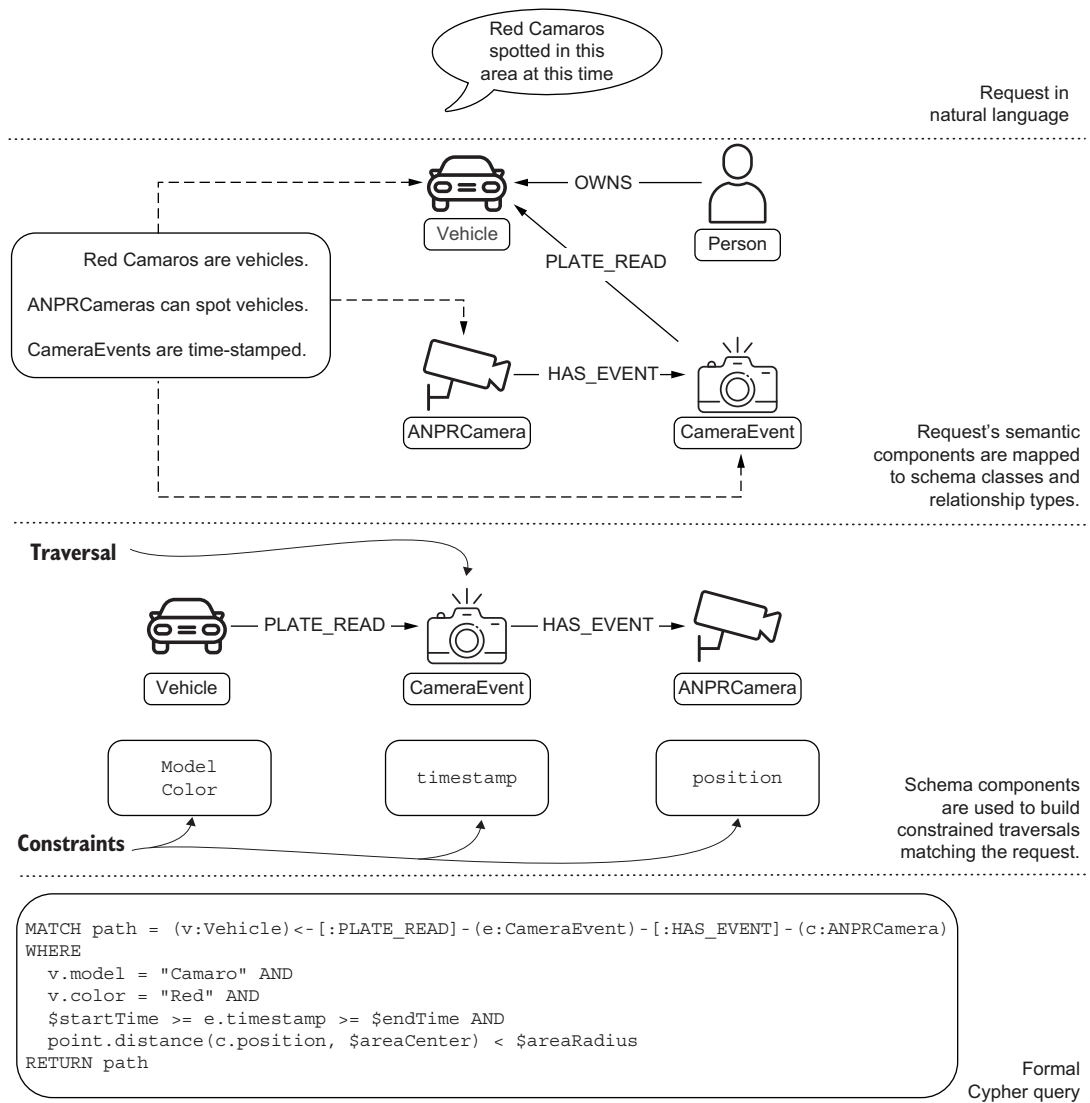


Figure 14.2 Translating a natural-language query (“Red Camaro spotted in this area at this time”) into a formal Cypher query. The translation occurs in three stages: (1) parsing the natural-language request into semantic components, (2) mapping these components to schema elements (**Vehicle**, **CameraEvent**, and **ANPRCamera** nodes with their relationships), and (3) constructing a formal Cypher query with the appropriate traversal patterns and constraints. Domain concepts are systematically transformed into graph database operations, demonstrating the bridge between user intent and executable queries.

14.4 **Think like an expert: Using metadata for enhanced querying**

In the previous section, we explored how experts approach querying a KG by using their understanding of the schema to construct precise, context-aware queries. This process, grounded in constrained traversals, serves as a blueprint for extracting specific insights from complex data structures. However, this expert-based approach is not only the domain of human experts. With the advancements in LLMs, it is possible to translate this expert reasoning into a set of tasks that an LLM can perform, effectively allowing the model to think like an expert.

Expert reasoning, as we have outlined, involves a systematic approach to understanding and navigating the schema of a KG. LLMs, which are trained on massive amounts of text data, can recognize patterns and make inferences that closely resemble expert reasoning. They can parse and interpret the schema, understand how different entities and relationships interconnect, and use this understanding to inform their reasoning process. This combination of reasoning and schema comprehension allows LLMs to generate more accurate and contextually appropriate queries, mirroring the expert's ability to navigate the graph's structure.

In the following sections, we'll examine the core steps involved in answering questions using KGs:

- First we'll explore how to understand the intent behind the question (section 14.5). This step is vital because it helps route the request to specialized pipelines, if necessary, ensuring that the most appropriate method is used to answer the query.
- Next, we'll discuss how to extract and use metadata—essentially, an enriched or annotated schema—that provides the LLM with a deeper understanding of the graph (section 14.6). This metadata acts as a guide, enabling the model to navigate the KG more effectively.
- We'll also examine techniques to mimic expert reasoning in the LLM (section 14.7). Although there are intrinsic limitations in an LLM's reasoning capabilities, we'll explore strategies to overcome some of these challenges, bringing the model's performance closer to that of a human expert.
- Additionally, we'll explain how to integrate a summarization step to highlight the most important parts of the answer (section 14.8). This process not only helps in distilling the key information but also enables the inclusion of valuable insights.

Figure 14.3 gives an overview of our system's architecture and the interaction between its core components. We'll use this diagram as our roadmap to understand how the elements work together to process and transform user questions into meaningful results.

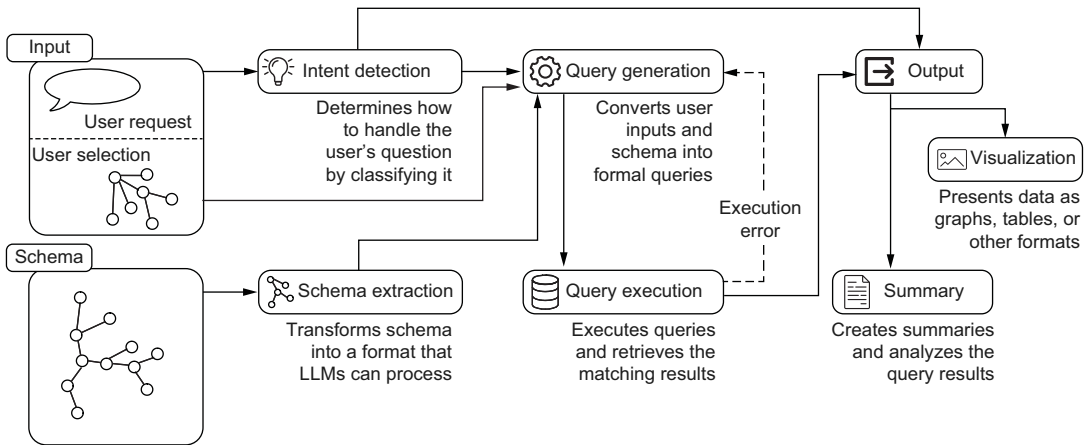


Figure 14.3 Overview of the system, highlighting the main components: Intent detection, Schema extraction, Query generation, Query execution, Visualization, and Summary generation. The system processes user questions and selection inputs while supporting error feedback during query execution.

Before we move on, it's important to recognize a significant paradigm shift in how we approach question-answering systems: shifting the focus from *generating an answer* to *asking the right question*. In traditional RAG systems, the process involves building a context around a question so that a grounded answer can be generated from the combination of the question and the context. The emphasis is on how to craft an answer that is both relevant and accurate based on the provided context.

However, in the approach we're exploring, the focus shifts toward converting questions into formal queries. Here, the challenge is not just answering the question but properly formulating the question so that it can be directly used to extract factual data from the KG. Although both questions and queries aim to uncover information, queries are essentially formalized questions designed to interact with the structure of the KG, ensuring precise and factual retrieval of data.

14.5 Intent detection: Understanding user expectations

Let's dive into our system architecture by exploring the first component: the Intent detection module (see figure 14.4). Why do we need to understand the intent to properly answer a question, and why should we do this at the beginning?

To answer the first question, we can revisit our “red Camaros” example. How should the answer be presented to the user? One option is to create an investigation board similar to those seen in crime movies, where we can add nodes and relationships. In this case, we would include not only nodes representing cars but also connections to the ANPR cameras that detected each vehicle. Alternatively, we could use a table representation that returns a list of cars with the plate number, camera location, and time of capture. Another option might be to draw a map, which would require the locations of the camera nodes. Each of these presentation methods has advantages, depending on the

user's needs and preferences. We need to understand the user's intent so that we can identify the appropriate type of presentation and the right pipeline or chain to choose.

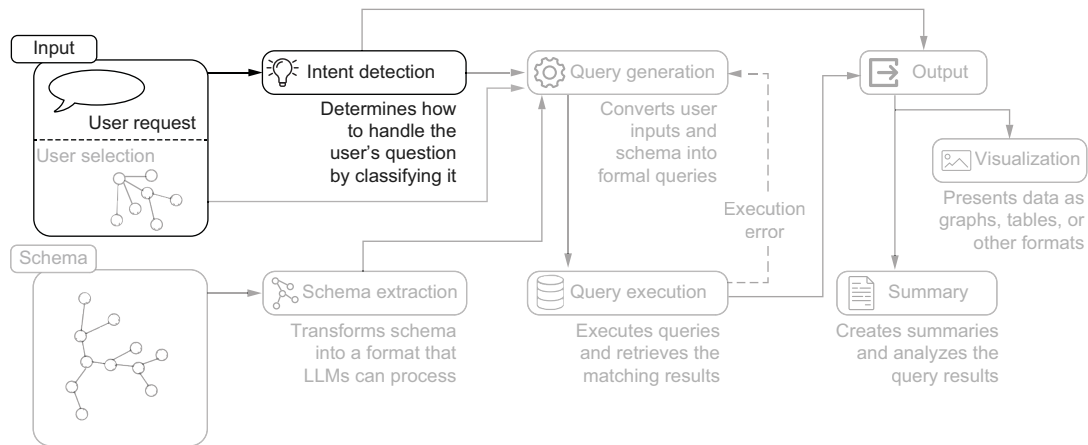


Figure 14.4 The Intent detection component analyzes user inputs to determine how to appropriately handle and classify the user's question.

We perform this step at the beginning for a couple of reasons. First, we aim to mimic expert reasoning, and understanding the intent behind a question is a fundamental first step that all experts take to provide satisfactory answers to their users. Second, this classification step relies on semantic understanding, a task that LLMs excel at. Because it is a relatively straightforward task, it is easy to follow and debug, providing a solid foundation for our question-answering system.

14.5.1 Classifying by visualization type

To build a classifier for intent, we must decide on the set of classes any question should fall into. This set is not written in stone; we can revisit it by adding or merging classes as we develop the system and as the system evolves in production.

Suppose our question-answering system is feeding a front-end layer capable of presenting the following:

- Graphs as interactive canvases with nodes and relationships laid out
- Tables with basic sort and search functions
- Charts or plots
- Interactive maps in a geographic information system (GIS)-like environment

As shown in figure 14.5, this creates a clear mapping between user intent and our system's visualization capabilities, forming the foundation for our intent detection approach.

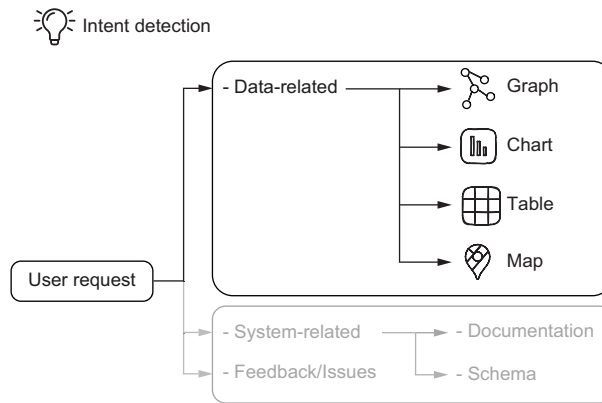


Figure 14.5 Intent detection system architecture for data visualization requests, showing how user requests are mapped to appropriate visualization formats (graph, chart, table, or map)

In this case, we can start by using a prompt to infer the type of representation the user wants to receive; this will be our intent detection, or at least the first version of it. A good classification prompt should have these characteristics:

- *Clear instructions*—The task is explicitly stated at the beginning, explaining what needs to be done.
- *Defined categories*—The categories are clearly defined so the model knows how to differentiate between them.
- *Examples*—By showing the model a few examples, it gets a clear idea of how the classification should work.
- *Boundary cases*—The prompt includes examples that sit near the boundaries between categories to help the model understand the nuances. This is especially important in classification, where similar inputs may belong to different categories.
- *Expected output format*—This guides the model on how to present the answers, ensuring consistency.
- *Fallback options*—If applicable, include a special category as a fallback, and instruct the model on when to use it. This can prevent forced or incorrect classifications.

The following example prompt implements such a classification task:

AN

Given a Text delimited by triple backticks representing a user question, identify the best output of the presentation.

Select one of the possible outputs in the following list:

"graph", "table", "chart", "map".

The first step is to understand if the user explicitly asks for a specific output type to show the results.

For example, if the user asks for graph elements such as paths or nodes or relationships, then the output must be a graph in any case.

If the output type is not explicit it is usually "graph":

- "table", only when the user asks about aggregation, ordering, and statistics;
- "chart", if the user asks for plotting distributions;
- "map", if the user asks for showing locations, places or other entities with a strong location property;

If you do not understand the output from previous cases the output should be "graph".

Here you can find some examples:

Example: Location of last 10 narcotics related crimes

Output: {"type": "map", "reason": "type is map because it involves showing locations"}

Example: Distribution of crimes over time

Output: {"type": "chart", "reason": "type is chart because a distribution can be plotted"}

Example: "Maximum, minimum, and average number of crimes per district"

Output: {"type": "table", "reason": "type is table because aggregations are requested"}

Example: "People involved in or related to crimes investigated by Inspector Morse"

Output: {"type": "graph", "reason": "type is graph because entities and relationships are implied"}

The output must be in JSON format. Do not explain the result.

###Text:````{{question}}````

###Output:

The classification prompt is divided into three sections:

- *Task definition*—The first sentence defines the task generically, and other details are added in subsequent sentences. We introduce a clear bias toward the graph representation, and then we describe the expectation for each class.
- *Few-shots section*—We provide a set of examples, including the expected responses. The format of the response is also defined because the output structure is simple and it is easier to show than to explain. The examples are in the context of the law enforcement domain, and the response includes a reasoning step in addition to the answer.
- *The actual question*—Finally, we insert the question, following the format we provided in the few-shots section above.

The following are some examples of questions and classification results obtained using the previous prompt:

-  What is the location of the latest shooting?
-  {"type": "map", "reason": "type is map because it involves showing locations"}
-  How many crimes were committed per month in 2020 compared to 2019?
-  {"type": "chart", "reason": "type is chart because a distribution can be plotted"}
-  Who are the main suspects in the recent string of burglaries?
-  {"type": "graph", "reason": "type is graph because entities and relationships are implied"}
-  How many traffic fatalities occurred last year compared to the previous year?
-  {"type": "table", "reason": "type is table because aggregations are requested"}

We've done a decent job—our prompt meets most of the criteria we listed earlier. We are now able to label each question with an intent that matches the type of output we can present.

NOTE You may wonder about the purpose of the `reason` field. The upcoming sections discuss how generating extra tokens affects the overall response of the LLM and how to exploit this knowledge to improve the quality of the results. For now, we are not planning to use the `reason` field in any downstream processing, but we will use it for debugging.

Next, let's take it up a notch by focusing on boundary cases and fallback options. The easiest way to obtain high-quality samples on the boundary is to observe misclassification during normal use. Consider this example: "What are the most common types of organized crime involved in human trafficking?" This question may be classified as a "graph" type because "entities and relationships are implied," even if it requires some aggregation and summarization.

On the one hand, the question is related to law enforcement, which often implies a need for visual representations of entities and relationships (e.g., networks of organized crime groups). On the other hand, the question asks for "common types" of organized crime, which suggests that some aggregation and summarization are required.

In this case, when we used our prompt, the system classified the question as a "graph" because the relationship between the different types of organized crime and human trafficking was more important than simply listing the common types, but it is not a clear-cut classification. There is no single correct and undebatable answer—but

these are the types of questions we want in the few-shot section, as they will help shape the nuanced behavior of our system.

We took some liberties regarding fallback options criteria and included a significant bias toward the “graph” type, which effectively serves as a “catch-all” class. This is a reasonable choice; we are querying a graph database, after all.

14.5.2 Is it data, documentation, or just complaining?

To empower nontechnical domain experts to use the power of the KG, our solution must consider the broader spectrum of questions users may ask. Although the intent detection approach we’ve discussed so far is well-suited for handling data-driven queries, we also need to address more generic, open-ended inquiries that may not have a straightforward mapping to the graph database.

Consider, for example, the following questions:

- “Is it possible to export data from the system into a CSV file?”
- “How do I use the system to assess the risk level associated with a suspect?”
- “The system is too slow and keeps freezing. What can be done about it?”
- “It would be great if the system had [feature XYZ]. Can this be added?”

These questions are as legitimate as they are unexpected. They may not directly involve querying the KG, but they reflect real user needs and expectations.

Users interacting with a system like this are not limited to data-driven queries; they may also seek help with system functionalities, request explanations, or even provide feedback. It’s reasonable to expect a variety of question types, which can generally be grouped into three categories:

- *Data-related questions*—These are the questions we have addressed so far, involving querying specific information from the KG.
- *System-related questions*—These are about how the system operates or its capabilities, such as exporting data or assessing risk.
- *Feedback and complaints*—Users may express frustration or make requests for improvements.

Among system-related questions, we can further distinguish between two important subcategories:

- *Documentation-related questions*—These questions can be resolved by consulting the user documentation, the user manual, or help.
- *Schema-related questions*—These are technical questions related to how the KG is structured and the decisions made during data modeling.

We can broaden the scope of the intent detection to encompass a full range of possible intents, as shown in figure 14.6.

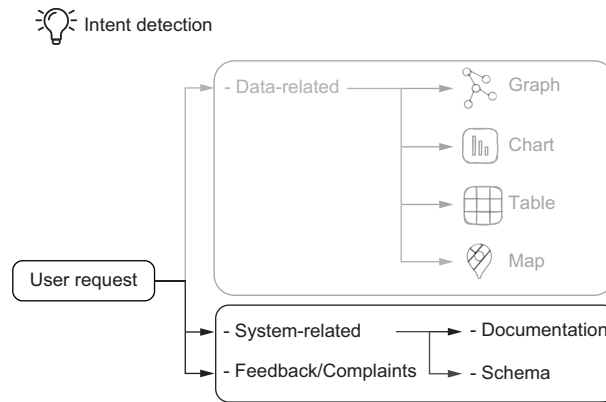


Figure 14.6 Classification of system-related questions in the intent detection system, showing how requests are routed either to documentation (for system functionality and feature questions) or to the schema (for KG structure queries). The system also identifies feedback and issues as a separate category for user complaints or enhancement requests.

With the range of potential user intents identified, the next critical step is crafting effective prompts for the LLM to handle this classification task. A well-structured prompt can guide the model in detecting the type of question and routing it to the appropriate pipeline—whether it’s a data-related request, a system-related question, or user feedback. The following prompt shows how this broader intent detection can be implemented:

AN

You are an AI assistant tasked with categorizing questions related to a law enforcement knowledge management system. Your job is to classify each question into one of three main categories:

1. ****Data-Related:**** Questions that require direct access to or knowledge of the data within the system in order to be answered. These questions focus on the content and structure of the data itself.
2. ****System-Related:**** Questions related to the system’s functionality, features, architecture, or design choices - information that would be found in documentation, not necessarily in the data itself.
3. ****Feedback/Complaints:**** Questions that are either not answerable by the system or are user feedback/complaints.

****If the question is classified as "System-Related,"**** you will further classify it into one of the following subcategories:

- ****Documentation-Related:**** Questions that can be answered by referring to system documentation, user manuals, or help sections.
- ****Schema-Related:**** Questions related to the structure of the knowledge graph or graph data schema.

****Task:**** For each question below, first classify it into one of the three main categories. If you classify it as "System-Related," then further classify it into either "Documentation-Related" or "Schema-Related."

****Example Questions and Expected Output:****

1. ****Question:**** "Is it possible to export data from the system into a CSV file?"

****Answer:****

```
{
  "question": "Is it possible to export data from the system into a
  CSV file?",
  "category": "System-Related",
  "subcategory": "Documentation-Related",
  "reason": "The question asks about the capability of the system
  regarding data
  export, which falls under system functionality."
}
```

2. ****Question:**** "How do I use the system to assess the risk level associated with a suspect or location?"

****Answer:****

```
{
  "question": "How do I use the system to assess the risk level
  associated with a suspect or location?",
  "category": "System-Related",
  "subcategory": "Schema-Related",
  "reason": "The question involves assessing risk, which requires
  understanding the graph schema and structure of the data related
  to suspects and locations."
}
```

3. ****Question:**** "What permissions do I need to access restricted data?"

****Answer:****

```
{
  "question": "What permissions do I need to access restricted
  data?",
  "category": "System-Related",
  "subcategory": "Documentation-Related",
  "reason": "The inquiry is about user permissions, indicating a
  question about system features and access controls."
}
```

4. ****Question:**** "Why isn't the system responding to my queries?"

****Answer:****

```
{
  "question": "Why isn't the system responding to my queries?",
  "category": "Feedback/Complaints",
  "reason": "This question expresses a complaint about system
  performance, rather than asking for specific data or features."
}
```

```

5. **Question:** "What types of data visualizations can the system
generate?"

**Answer:**
{
  "question": "What types of data visualizations can the system
generate?",
  "category": "System-Related",
  "subcategory": "Documentation-Related",
  "reason": "The question seeks information on available features
related to data visualizations, which pertains to system
documentation."
}

**Begin classification:**
{{question}}

```

This classification prompt is divided into three sections: the *task definition*, a *few-shot section*, and the *actual question*. Additionally, the expected output format includes a *reason* field, which is important for identifying potential misclassifications and understanding the model's reasoning.

Let's look at a misclassification example. Suppose we ask "How often is the knowledge graph updated to reflect the latest information?" A smaller or quantized LLM responded this way:



```

"question": "How often is the knowledge graph updated to reflect the
latest information?",
"category": "Data-Related",
"reason": "The question concerns the frequency of updates within the
data itself."

```

But a full-scale LLM gave this response:



```

"question": "How often is the knowledge graph updated to reflect the
latest information?",
"category": "System-Related",
"subcategory": "Documentation-Related",
"reason": "The question is asking about the update frequency of the
knowledge graph, which is a system-related feature that would be
documented."

```

In this example, the question is related to how frequently the KG is updated. Although it's possible to infer this information by analyzing data, such as the creation or update timestamps of nodes, this approach would not be appropriate for a law enforcement knowledge management system. Attempting to determine the update frequency by analyzing the data may not provide an accurate conclusion, as the data could reflect how the information happens to be entered rather than how the updates are scheduled.

Larger models can detect that this question pertains to system functionality and refer to the documentation, labeling it as "System-Related" under "Documentation

-Related.” Smaller or quantized models, however, may misinterpret the question as concerning the data itself and incorrectly classify it as “Data-Related.”

The content of the `reason` field in both cases provides valuable insight. It suggests that the smaller model lacks context. This signals that we should enhance the task definition section by including relevant background information, helping guide the model to the correct classification.

To assess how well our prompt performs, we can evaluate it based on the criteria we established in the previous section and see if it includes clear instructions, defined categories, boundary cases, examples, and expected output format.

Design considerations for classification prompts

As we consider the design of our prompt for intent detection, it’s essential to weigh the trade-offs between using a single broader prompt and implementing a multistage classification approach. This consideration applies to classification tasks in general.

Opting for a *single broad prompt* can be beneficial if simplicity and reduced management overhead are priorities. This method streamlines the classification process and can be effective when the model’s accuracy is acceptable even for more complex tasks. It allows for quicker implementation and easier maintenance, particularly in environments where rapid deployment is necessary.

On the other hand, a *multistage approach* may be suitable if accuracy and flexibility in classification are more critical or if you anticipate frequent adjustments to how questions are categorized. This method provides more granular control over the classification process, enhancing the handling of complex questions and edge cases. By breaking the classification into stages, you can refine each step according to specific requirements, ultimately leading to more robust and reliable results.

Starting with a single broader prompt maintains simplicity while letting us evaluate the prompt’s effectiveness in practice. As the complexity of requests increases or new requirements emerge, transitioning to a multistage classification system can be considered.

Postponing the decision to split the prompt into multiple stages also provides the opportunity to gather real-world examples. These cases can be invaluable in refining the classification process, ensuring that the eventual multistage approach is tailored to meet the specific needs of our application.

14.6 From schema to LLM-ready context

As shown in figure 14.7, we’re focusing on the schema extraction phase of our pipeline. This critical step forms the foundation for transforming raw schema information into formats that LLMs can effectively process and understand.

Now we will build on this foundation by exploring how metadata, an enriched layer of schema information, can empower LLMs to replicate this expert reasoning, ultimately enabling nontechnical users to query KGs effectively. Here, the enrichments are those that would be beneficial to experts, as anything that makes the schema more understandable or actionable for an expert can be exploited by our LLMs.

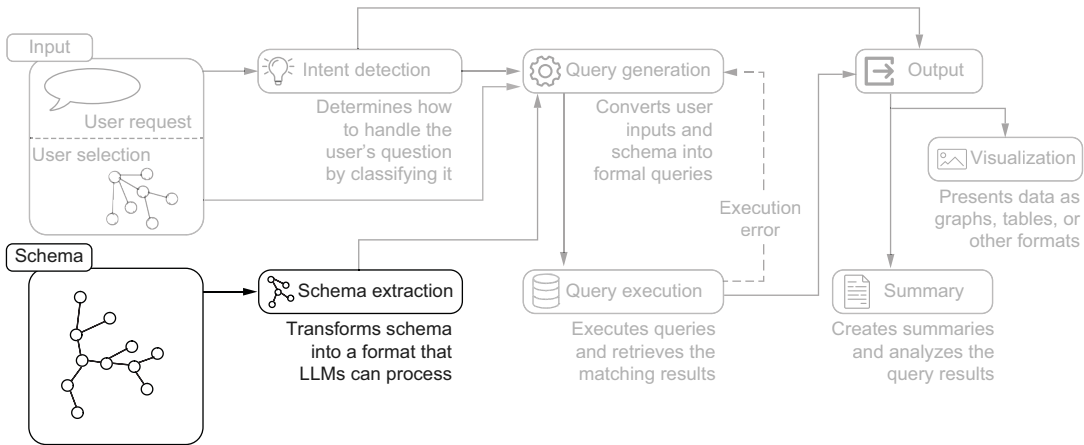


Figure 14.7 Schema processing pipeline, highlighting the transformation of the database schema into LLM-compatible formats. The diagram illustrates how raw schema structures are processed through schema extraction to create representations that LLMs can effectively process.

14.6.1 Schema extraction and representation

The challenge we face is extracting the schema from the KG and converting it into a form that can be processed by a language model. We can then use that schema information to help the language model better understand the question and produce a proper Cypher query.

At first glance, we could solve the first problem by invoking the `apoc.meta.schema` [1] function, which computes the schema based on the graph's current structure.

Listing 14.2 Response format for `apoc.meta.schema`

```
[
  {
    "label": "[LabelName]",
    "properties": {
      "[PropertyName1]": {
        "type": "[PropertyType1]",
        "mandatory": [true|false],
        "unique": [true|false]
      },
      "[PropertyName2]": {
        "type": "[PropertyType2]",
        "mandatory": [true|false],
        "unique": [true|false]
      },
      ...
    },
    "relationships": [
      {
        "type": "[RelationshipType]",
        "target": "[TargetLabelName]",
        "properties": {
```

```

    "[RelationshipPropertyName]": {
      "type": "[RelationshipPropertyType]",
      "mandatory": [true|false],
      "unique": [true|false]
    }
  },
  ...
]
},
...
]

```

However, a closer look reveals that the schema provided by this method is a *technical database schema*. It includes many details irrelevant to our needs, such as helper or administrative nodes, technical metadata properties, unnecessary or redundant type labels, unused relationships, and unused properties. These extraneous details clutter the schema with elements that are useful for database management but do not contribute to the core understanding of the domain. To address this, we need a *conceptual KG schema*: a distilled, meaningful structure that focuses only on the entities and relationships which convey the essential model of the domain.

The *conceptual schema* serves as a simplified, domain-relevant subset of the technical schema, stripping away technical complexities and irrelevant metadata. By removing auxiliary elements, the conceptual schema prioritizes clarity and usability, capturing the logical relationships and characteristics that experts rely on when reasoning about the graph.

This simplification is important for several reasons:

- *Alignment with human reasoning*—The conceptual schema reflects how domain experts understand the KG, using only the relevant entities, relationships, and properties to answer domain-specific questions. This aligns the schema more closely with the natural language model's approach to reasoning, making it easier to map questions accurately to graph elements.
- *Reduced cognitive load for LLMs*—Language models, particularly LLMs, are more efficient when given focused, relevant data rather than the full, complex structure of a technical database schema. By providing a streamlined conceptual schema, we allow the model to concentrate on meaningful information, increasing its ability to generate accurate and relevant Cypher queries without being distracted by extraneous details.
- *Minimized risk of query errors*—Technical schemas often contain implementation-specific elements, redundant labels, or metadata that could confuse the model, potentially leading to errors in query generation. The conceptual schema eliminates this noise, reducing the likelihood of misinterpretations that could result in incorrect queries.
- *Enhanced model interpretability*—A conceptual schema presents the KG in a more human-readable form, aligning it with how LLMs are trained to interpret text. By

focusing on the essential model of the domain, the conceptual schema captures the structure that LLMs can most effectively use to infer intentions and mappings.

To transition from the raw output of `apoc.meta.schema` to a conceptual schema, some human intervention is necessary. One approach is to describe the conceptual model manually, effectively “curating” the schema; alternatively, we can define a skip list to filter out unneeded elements from the APOC results, thereby distilling the technical schema to its conceptual core.

To make the conceptual schema interpretable for a language model, we must represent each node class and relationship in a clean, structured format that focuses on the essentials. This approach ensures that the LLM has a clear understanding of core classes and connections, as well as their properties and types, without the distractions of technical details. Figure 14.8 illustrates the key steps in transforming a technical Neo4j schema into an LLM-friendly conceptual representation.

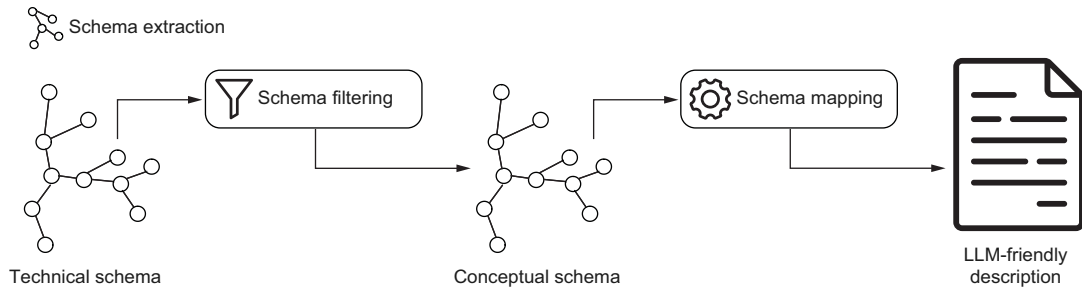


Figure 14.8 The technical schema obtained through the APOC call is filtered so it is reduced to the conceptual graph schema representation. The conceptual schema is then mapped into a textual format that can be effectively understood by LLMs.

In a *narrative schema representation*, nodes and relationships are described in greater detail and natural language, including rich context and examples (listing 14.3). Although this format is useful for human readers who need a thorough understanding, it can be more challenging for a language model to process when the goal is efficient query generation.

Listing 14.3 Narrative schema representation

```

**Nodes:**

1. **Vehicle**:
  - **Properties**: `color`, `make`, `model`, `style`, `plate_number`
  - Node example:
    - (:Vehicle
      {make: "Toyota", model: "Camry",
       style: "Sedan", plate_number: "XYZ123"})`
  
```



```

**Relationships:**

1. **OWNED_BY**: Represents the relationship of ownership between a vehicle
   and a person.
   - Example:
     - (:Vehicle {plate_number: "XYZ123"})
       - [:OWNED_BY] ->
         (:Person {name: "Alice"})

```

As the schema grows, this approach can be verbose and less systematic. For a language model, this kind of representation may introduce challenges in parsing and understanding, particularly when the schema becomes large and complex.

In contrast, a more LLM-friendly format distills the schema into a concise, consistent structure that focuses on key entities and relationships, stripping away extra verbiage while retaining the essential details.

Listing 14.4 LLM-friendly schema representation format

```

Nodes:
(:Vehicle {
  color: STRING,
  make: STRING,
  model: STRING,
  style: STRING,
  plate_number: STRING
})

Relationships:
(:Vehicle)-[:OWNED_BY {since: DATE}]->(:Person)

```

This structured representation includes just the essential components—entity names, properties, and relationships—without extraneous descriptions or complex formatting. It is easy for the LLM to process because it uses a standardized syntax that aligns well both with the entity types and their attributes. The relationships are described clearly with property types and minimal but necessary contextual information.

14.6.2 *Enriching schemas with descriptive annotations*

In the previous section, we introduced schema descriptions as a way to bridge the gap between the KG’s structure and the LLM’s ability to generate accurate, meaningful queries. However, relying on the schema structure alone can hinder this process.

Suppose, for instance, that the user asks to “find all black vehicles involved in a crime.” The LLM might naively translate this to `Vehicle.color == "black"`, assuming “black” is stored in a readable string format. However, if the database uses abbreviations like `BLK` for “black,” this query will miss all relevant records, giving the impression that no black cars were involved in any crimes.

Another similar issue may arise with relationship ambiguity. Suppose the KG includes both the `COMMITTED` and `CO_OFFENDS_WITH` relationships. If the user asks to

“list all suspects who have been in multiple crimes with another person,” the LLM has no clear basis for determining which relationship to use. Is the question about those directly involved (COMMITTED) or individuals with co-offending histories (CO_OFFENDS_WITH)? Lacking a clear distinction, the LLM may choose incorrectly, amplifying the inherent ambiguity of natural language and leaving the user feeling as if they’re communicating with a system that doesn’t fully grasp their request.

To tackle these issues, we can mimic the actions an expert would take in similar situations. When faced with unfamiliar data, an expert begins by familiarizing themselves with the KG’s structure, consulting any available documentation and data dictionaries. This careful review helps them understand the correct paths to take through the data and allows them to set constraints with precision, avoiding the pitfalls of ambiguous or misinterpreted terms.

As part of this familiarization process, experts may create a kind of “cheat sheet” to capture key insights—such as terminology, abbreviations, and relationship meanings—that aid in accurately constructing queries. For example, they might note that “BLK” is used instead of “black” in the vehicle color property, or that specific relationship types like COMMITTED and CO_OFFENDS_WITH capture involvement in criminal activities from different angles. By capturing this information, experts avoid mistakes and ensure that their queries align with the KG structure and semantics.

We can replicate this process of building a cheat sheet by systematically annotating the KG’s schema, documenting node classes, relationship types, and properties with descriptions that capture these nuances. This annotated schema becomes a guide for the LLM, allowing it to make more informed and contextually accurate query translations. An example of such an annotated schema representation is shown in the next listing.

Listing 14.5 Annotated schema representation format

```
Nodes:
/* Represents a vehicle involved in various incidents or owned by individuals
 */
(:Vehicle {
  color: STRING, /* Color of vehicle, BLK, GRY, SIL, WHI, etc*/
  make: STRING, /* Manufacturer: BMW, BUIC, CADI, CHEV, etc */
  model: STRING, /* Model of the vehicle: IMP, ALT, SON, SEB, CIV, etc */
  style: STRING, /* Body style: SUV, SEDAN, etc */
  plate_number: STRING /* Vehicle license plate */
})

Relationships:
/* Ownership relationship from vehicle to person, with start date of
   ownership */
(:Vehicle)-
[:OWNED_BY {since: DATE /* Date ownership began, ISO format */}]
->(:Person)
```

With these annotations as inline comments, the LLM has access to detailed descriptions that guide it in accurately interpreting the schema, leading to more precise queries which reflect the true data structure.

14.6.3 A practical approach to schema representation

Building on the previous discussions, we now propose a practical approach to streamline the process of refining and annotating the KG schema. This approach uses a YAML configuration file to manage the output of `apoc.meta.schema`, distill it by skipping irrelevant elements, and add rich descriptions to help LLMs understand the schema more effectively.

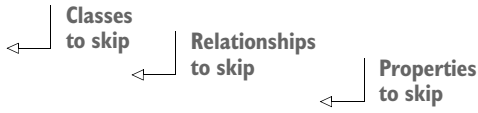
The YAML file serves as a central configuration point where users can specify which parts of the schema to exclude from processing, as well as provide detailed descriptions for key entities, relationships, and properties. The file includes two main sections: `skip` and `descriptions`.

SKIP LIST

The `skip` section allows users to filter out certain classes, relationships, or properties that are deemed not to be part of the core *conceptual KG schema* as we described it.

Listing 14.6 Schema config: `skip` section

```
schema:
  skip:
    classes: []
    relationships: []
    properties: []
```



The diagram illustrates the structure of the `skip` section. Three arrows point from the `skip` section to three labels: **Classes to skip**, **Relationships to skip**, and **Properties to skip**.

DESCRIPTIONS

The `descriptions` section is where detailed annotations are added for various schema elements. Here, users can describe the purpose and semantics of classes, relationships, and properties, helping the LLM interpret the graph more effectively. These descriptions can be as granular as needed, ensuring that both the LLM and any human users interacting with the schema have a clear understanding of its structure.

Listing 14.7 Schema config: `descriptions` section

```
descriptions:
  classes:
    Class1: "Description of class 1"
  relationships:
    Rel1: "Description for relationship type 1"
  properties:
    Class1:
      property1: "Description for Class1.property1"
      property2: "Description for Class1.property2"
      [...]
    Rel1:
      property1: "Description for relationship property Rel1.property1"
```

The benefits of this approach include the following:

- *Customization*—Users can easily adjust the schema representation by editing the YAML file to include only the necessary components and tailor the descriptions to their needs.
- *Maintainability*—The YAML file serves as a centralized, human-readable configuration that can be easily updated as the schema evolves, ensuring that the schema remains aligned with the needs of the domain and the LLM.
- *Scalability*—As the KG grows, this approach allows for a manageable and scalable way to handle schema modifications, ensuring that the LLM can keep up with new data and relationships without becoming overwhelmed by unnecessary complexity.

An example of the output from this schema configuration approach is shown next.

Listing 14.8 Schema description: output example

```
### Graph Schema Overview

#### Node Types
(:Vehicle /* Represents a vehicle involved [...] */ {
  color: STRING, /* Color of vehicle, BLK, GRY, SIL, WHI, etc*/
  make: STRING, /* Manufacturer: BMW, BUIC, CADI, CHEV, etc */
  model: STRING, /* Model of the vehicle: IMP, ALT, SON, SEB, CIV, etc */
  style: STRING, /* Body style: SUV, SEDAN, etc */
  plate_number: STRING /* Vehicle license plate */
})

#### Relationships
(:Vehicle) - [:OWNED_BY /*<description>*/ {since: DATE /* <description> */}] -
> (:Person)
```

14.7 It's time to think: Understanding LLM reasoning

Now that we have a structured understanding of user intent and an LLM-friendly graph schema, we're almost ready to tackle one of the most challenging steps: translating text into Cypher queries. Figure 14.9 illustrates how query generation acts as a convergence point, combining processed user intent and schema data, while also using execution error feedback to refine and improve query formulation through an iterative process.

Just as we'll learn how important it is to guide LLMs to “take their time” with reasoning, we shouldn't rush through prompt design. LLMs are designed to understand context and generate responses that seem remarkably human by using the immense amounts of text data they were trained on. However, when presented with complex or nuanced questions, LLMs may “rush” to a conclusion, relying on shortcuts in the data patterns they've observed rather than fully reasoning through the problem. To tackle this tendency, researchers have explored techniques like the following:

- *Chain-of-thought prompting* [2] introduces intermediate steps into the model's response generation. By structuring prompts that require the model to articulate

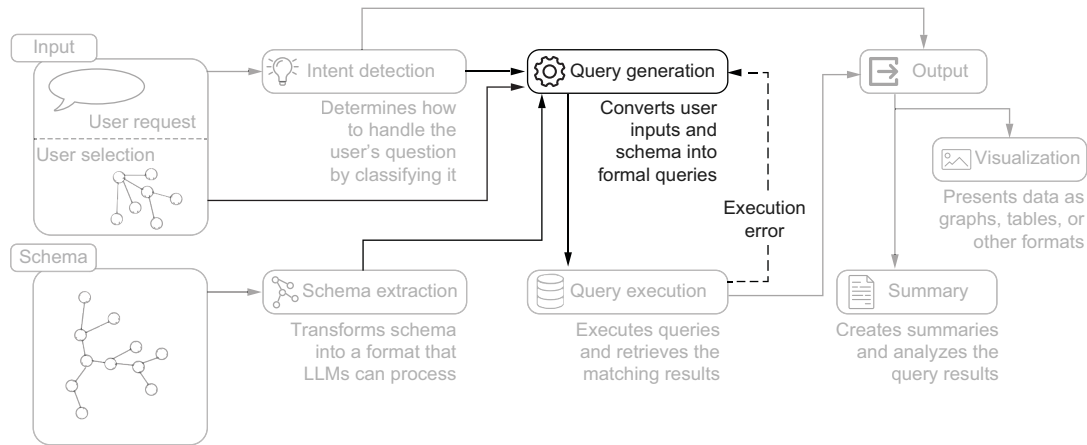


Figure 14.9 The query generation stage of the system architecture, highlighting its central role in converting user inputs and schema information into formal database queries

its reasoning step by step, this technique uses an LLM’s pattern-based capabilities, encouraging it to take a more calculated approach rather than jumping to conclusions.

- *Scratchpad techniques* [3] embed intermediate “workings” in the LLM’s output, where the model produces tokens that represent various steps or computations needed for reasoning through complex questions.

Both approaches give the model “time to think” by encouraging it to spend more computational resources on the problem-solving process before generating a final output. By modifying the prompt to “force” token-by-token generation before producing the final answer, these methods help allocate more computational capacity to each layer, effectively scaling the thought process in response to question complexity.

14.7.1 The order matters: Answer first vs. reasoning first

If an LLM is instructed to first provide the answer, followed by the reasoning behind it, the model may stick with the initial answer it generated. This is because LLMs typically produce one word (or, more precisely, one token) at a time, effectively committing to their previous choices as the generation progresses. This can lead to the model generating reasoning that is tailored to support the predetermined response, rather than reflecting an impartial, step-by-step thought process.

This consistency-driven tendency has been studied in recent research, which introduced the concept of *semantic consistency* [4]. The text corpora that these models are trained on typically exhibit a degree of logical coherence and internal consistency: what is said earlier is often used to support what is said later, and radical departures or contradictions are relatively rare. This consistency acts as a guardrail, preventing the models from engaging in what we would perceive as rambling speech.

However, this consistency can also become a constraint, due to the following problems:

- *Cumulative context*—Because the model generates tokens sequentially, the reasoning and context build cumulatively. This means the model's commitment to a particular answer can be influenced by the tokens it has already generated, reinforcing the idea of output consistency.
- *Error propagation*—If an error occurs early in the token generation process, it can propagate through subsequent tokens, as each new token is generated based on the previous ones. This highlights the importance of careful reasoning and validation at each step.

Figure 14.10 shows an example of how simply inverting the order of reason and answer generation can lead to significant differences in output.

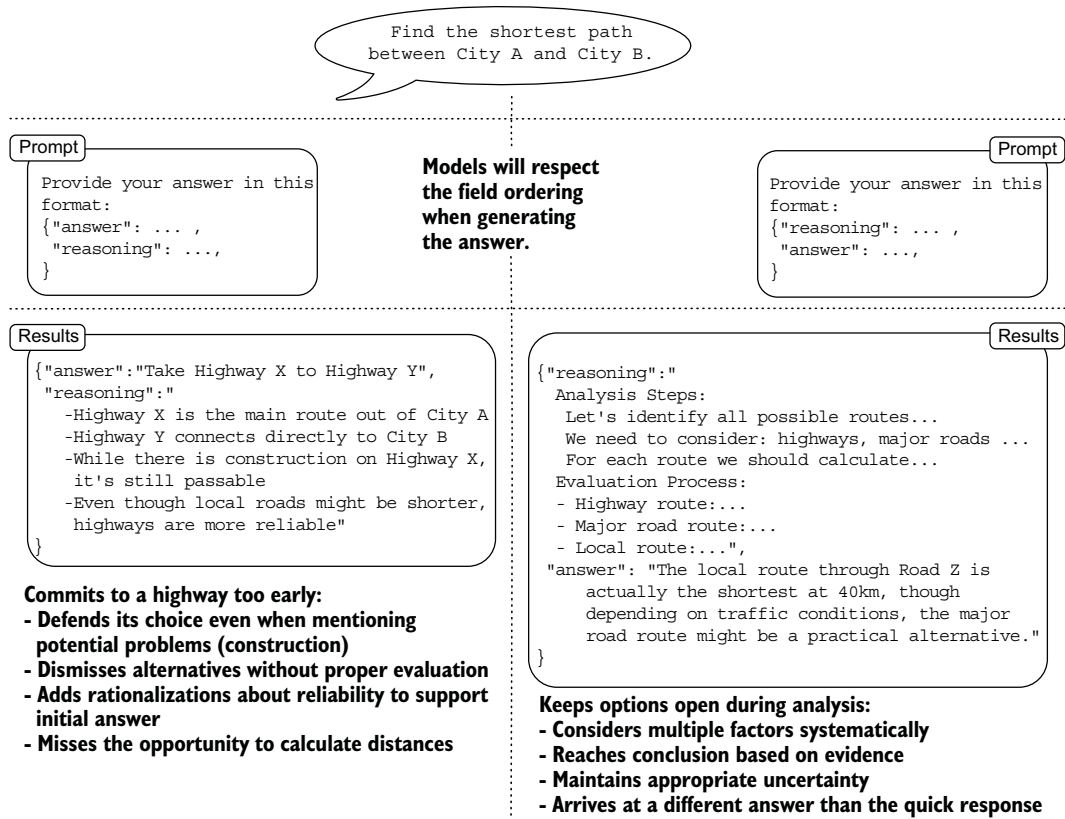


Figure 14.10 Comparison of two prompt structures for the same path-finding task. Left: The answer-first approach encourages quick, potentially biased responses with post hoc justification. Right: The reasoning-first approach promotes systematic analysis before reaching a conclusion. Note how the JSON structure in each prompt guides the LLM's thinking process.

To encourage transparent and reliable reasoning from LLMs, it's important to prompt the model to first provide its step-by-step reasoning, followed by the final answer. This structure forces the model to engage in a more thoughtful, deliberative process, as it must justify its conclusion through a balanced exploration of the problem, rather than simply rationalizing a predetermined response.

Postponing the reasoning step until after the answer can make sense in classification tasks. In these cases, we are more interested in understanding how the system justifies its choices, especially when it misclassifies. By using the reasoning provided by the model, we can gain insights into why the classification was incorrect and use that information to provide good examples that help clarify the classification boundaries, as we did in section 14.5.

14.7.2 *Thinking in queries: From text to Cypher*

Now that we've given both the model and ourselves plenty of time to think, it's time to put that thinking into action. Let's dive into crafting the prompt that will guide our LLM in generating the Cypher queries we need, using what we have built so far.

The prompt is structured as follows:

- A brief task description and question
- A schema definition with annotations
- Intent-dependent requirements
- Examples
- Optional user selection
- KG-specific annotations
- A reminder of the question
- A reminder of the requirements
- Output format specification

This structured approach forms a complete pipeline for transforming natural language questions into Cypher queries, as illustrated in figure 14.11. As we will see, the prompt includes elements we introduced in previous sections as well as new components.

BRIEF TASK DESCRIPTION AND QUESTION

We begin by introducing the task of translating a question in natural language into a Cypher query, anticipating that we will provide both the question and a detailed description of the schema. We use an HTML-like tag to wrap the user's question, which will help the model clearly understand the boundaries of the question and separate it from the rest of the instructions.

AN Your task is to generate a Cypher query for a Neo4j graph database, based on the schema definition provided, that answers the user Question.

The question we need to answer is:

```
<QUESTION>
{{ question }}
```

</QUESTION>
[...]

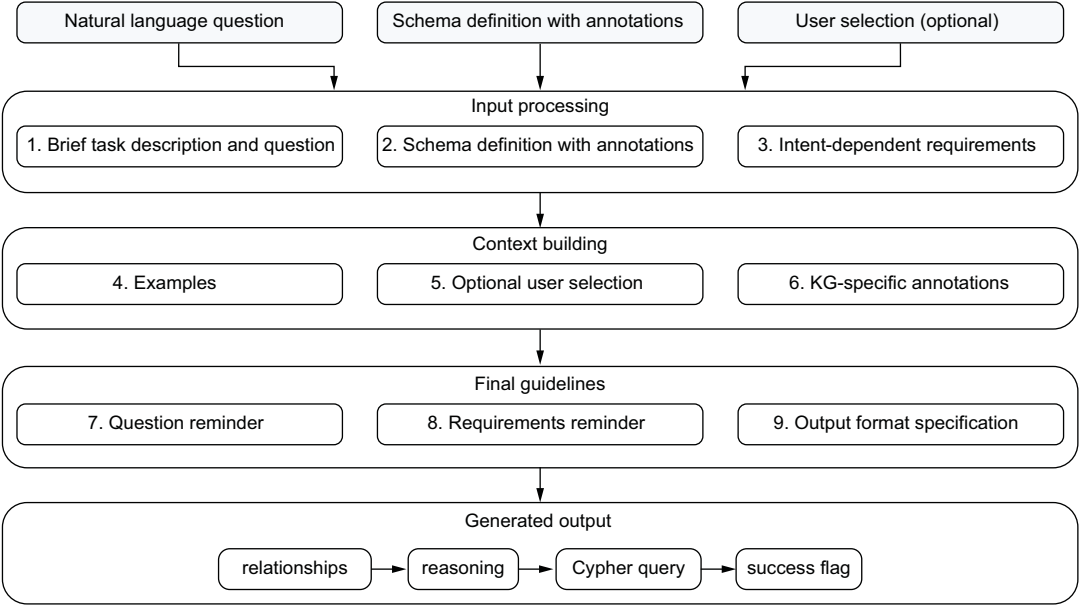


Figure 14.11 The complete flow of transforming a natural language question into a Cypher query, showing how three inputs are processed through structured prompt components. The process is organized into three stages (Input processing, Context building, and Final guidelines) that culminate in structured JSON output.

SCHEMA DEFINITION WITH ANNOTATIONS

In this section, we use what we developed during the schema extraction phase. The LLM-friendly format we introduced will seamlessly integrate with another HTML-like tag for this purpose. Using markdown such as `### Graph Schema Overview` or `#### Node Types` for headings will help the LLM create a clear structure for representing the schema. The `/* comments */` we added to annotate classes, relationships, and properties integrate the schema naturally.

AN [...]

The knowledge graph has the following schema, which the Cypher query must follow:
<SCHEMA>
{{ schema }}
</SCHEMA>
consider the comments as annotations
[...]

INTENT-DEPENDENT REQUIREMENTS

We use the output of intent detection to incorporate requirements that depend on the identified intent. In the case of a graph or map, we specify how the query should be structured. Specifically, we aim to retrieve not only the nodes that answer the question but also the relationships involved. For example, suppose we ask about red Camaros spotted by cameras in a certain area. We want to obtain information about the vehicles, the cameras, and the relationships that connect these elements. In case of a “table” response, we clarify that a tabular format is expected, so the model should select specific properties to display in columns rather than returning complete nodes or relationships.

AN [...]

```
{%if output_type == "graph" or output_type == "map" -%}
```

The result must be a graph so make sure to follow the schema and the following requirements:

- Return all the nodes and relationships matched, do not use anonymous relationships (such has (node0)-[:RELATIONSHIP]->(node1) instead use (node0)-[rel0:RELATIONSHIP]->(node1)

- Aggregate multiple traversals in a single MATCH pattern if possible: ``MATCH path=(p:Person)-[acted:ACTED_IN]->(m:Movie)<-[directed:DIRECTED]-(d:Director) RETURN path`` instead of ``MATCH path=(p:Person)-[acted:ACTED_IN]->(m:Movie), (d:Director)-[directed:DIRECTED]->(m)``

```
{%-endif%}
```

```
{%if output_type == "table" -%}
```

The result must be a table, i.e. you must select nodes and relationship properties and rename them to be presented in a table

```
{%-endif-%}
```

[...]

EXAMPLES

This is a new component, although we introduced the few-shots technique previously. The idea is to assist the system in answering foundational questions that can serve as building blocks for more complex queries, while also demonstrating the expected format in a single instance. Examples are a powerful and relatively inexpensive way to enhance the quality of responses in many situations.

AN [...]

Use only the provided labels, relationships, and properties; do not use anything else that is not specified.

If you cannot generate a Cypher statement based on the provided schema, explain the reason to the user.

{{examples}}

You must respect relationship types and directions.

[...]

OPTIONAL USER SELECTION

This is another new component. If the system can represent nodes and relationships, it will probably also let users select nodes. Incorporating the current selection into the prompt enables users to refer to it in their questions. Users may choose to refer to the selected node as a whole or its properties, as in “Give me the older siblings of the selected person.” The selected nodes are represented as a list of items, each containing the node label and its properties. If the user references a selection that is empty, the model highlights the issue.

AN

[...]

{%if selection -%}

Current selection:

{% for node in selection%

- {{node.label}} node with this properties {{node.properties}}

{% endfor %}

{%-else-%}

The selection is currently EMPTY. If there are references to selected nodes in the question, it is almost

certainly an error and therefore it is not possible to generate a response.

In this case, 'success' should be false.

{%-endif-%}

{{annotations.notes if annotations.notes}}

Do not include any explanations or apologies in your responses.

[...]

KG-SPECIFIC ANNOTATIONS

In this section, users can include clarification notes that are relevant to the specific KG. These annotations provide context and enhance understanding, helping the model interpret the schema more effectively. By not integrating these notes directly into the prompt, we ensure that the prompt can be reused across different KGs. This flexibility

allows for a more adaptable approach to query generation, as the annotations can be tailored to the specific nuances of each KG without altering the core prompt structure.

AN [...]

```
{{annotations.notes if annotations.notes}}
```

Do not include any explanations or apologies in your responses.

[...]

REMINDER OF THE QUESTION

We repeat the question near the end of the instruction prompt to reinforce the context and intent, which can lead to a more focused response. The schema definition section can be quite long, so the question may be located many tokens from the end of the instructions and the start of the response.

AN [...]

The question we like to answer may have some information that is relevant for the Cypher query:

```
<QUESTION>
```

```
{{ question }} {{ information }}
```

```
</QUESTION>
```

```
{%if output_type == "graph" or output_type == "map" -%}
```

Remember the requirements:

- Return all the nodes and relationships matched, never use anonymous relationships (ie `[ :RELATIONSHIP ]`), always use named ones (ie `[ rel1:RELATIONSHIP ]`).
- Aggregate multiple traversals in a single `MATCH` pattern if possible

```
{%-endif%}
```

[...]

Although language models, particularly those based on transformer architectures, do not inherently prioritize local context over global context—meaning they can theoretically focus on both local and distant tokens—these models can learn to give more weight to nearby tokens if they are more relevant for a particular task.

At this stage, we also provide an optional `information` field that can be relevant for generating the query. This field can be used, for example, to report a previous failure of the generated query, including the error message returned by the database. This allows the model to review its previous decisions and generate an error-free query in subsequent attempts.

OUTPUT FORMAT SPECIFICATION

Finally, we specify the format of the response, which must be a valid JSON object so that we can easily process it programmatically. We define each field we expect, provide hints for the expected value types for these fields, and include comments about the type of content we expect the model to fill in for these fields.



[...]

Use the "reasoning" field to explain your plan for the cypher query

Answer only in valid JSON in the following JSON format, nothing else (no <ANSWER> tags or anything like that):

```
{
  "relationships": [...], list of relationships to traverse, empty if not traversal is
  needed
  "reasoning": "...", this is the scratch pad for your reasoning
  "query": "<Cypher query>", must be a string and a valid Cypher query.
  "success": <true/false>, where true means that a Cypher query (following the
  schema) was returned.
}
```

14.7.3 Structuring output for reliable query generation

Let's dive deeper into the format of the output and the reason to select each field in a specific order. We start by asking the model to list the relationships it believes it should traverse. We do this because LLMs may hallucinate relationships that we didn't mention in the schema description; asking it to list the relationships it intends to traverse "out loud" significantly reduces the chance of this type of hallucination. Depending on the type of model, we may have to do some tuning. For example, if the model doesn't hallucinate relationships, we can safely remove this part. On the other hand, if the model commits too early to the relationships to traverse, we can soften the requirement for these fields by asking the model to list the relationships it will "potentially traverse," for example.

We then ask the model to generate the `reasoning` field. This gives the model time to think through the answer by breaking down the problem, defining a plan, and so on before committing to an answer. If you want to drive the reasoning process more precisely for specific use cases, you can add extra details under the "Use the "reasoning" field to explain your plan for the cypher query" section of the prompt, as well as through the examples.

We finally ask the model to produce the Cypher query to answer the question. Because we ask the model to generate this field after the `relationships` and the `reasoning` field, it will try to be coherent and not use nonexistent relationships. By guiding the model through the step-by-step process of identifying relationships, reasoning through the problem, and then formulating the final query, we ensure that the

output is well-grounded and aligned with how a human expert would tackle the task. This helps us get results that are more reliable and trustworthy compared to simply asking for the Cypher query directly.

ENSURING EXAMPLE CONSISTENCY

To ensure that examples don't undermine this structured approach, they must conform to our response format to avoid introducing ambiguity during response generation:

```
AN {%for example in examples%}
  Example:
  <QUESTION>{{example.question}}</QUESTION>
  {
    ...
    "query": "{{example.answer}}"
    {{ "'reasoning':" + example.reasoning + "'" if example.reasoning else "..."}}
  }
{%endfor %}
```

We use the "..." notation in the examples as a placeholder, indicating fields where we want to give the model the freedom to generate content autonomously. If we want to include important information in the reasoning step without significantly influencing the model's reasoning, we can run a prompt that asks the question in the example without including the `reasoning` field. This gives us the chance to check how the model would tackle the reasoning step without that guidance, and then we can modify the prompt to add the critical reasoning content we want to include without radically changing the overall response.

14.8 *Response summarization: From results to insights*

Having established a robust system that transforms natural language questions into actionable Cypher queries, our focus now shifts to ensuring that the results are as accessible and insightful as the queries themselves. Visualizing data as graphs provides a powerful way to explore relationships and structures, but it can sometimes leave users seeking clarity or deeper context. This is where summarization comes into play (see figure 14.12).

The summarization step holds a unique position in our pipeline as the first and only component that has access to the actual data the user is seeking. Whereas the previous steps focus on understanding the question and formulating the right query, summarization bridges the gap between raw data and user understanding.

Graph visualization excels at showing relationships and structures, but valuable information often lies in node properties or in the broader context of the results. A well-crafted summary can highlight these findings that may not be immediately visible in the visual representation. It provides users with a quick overview before they dive into a detailed exploration of the graph itself.

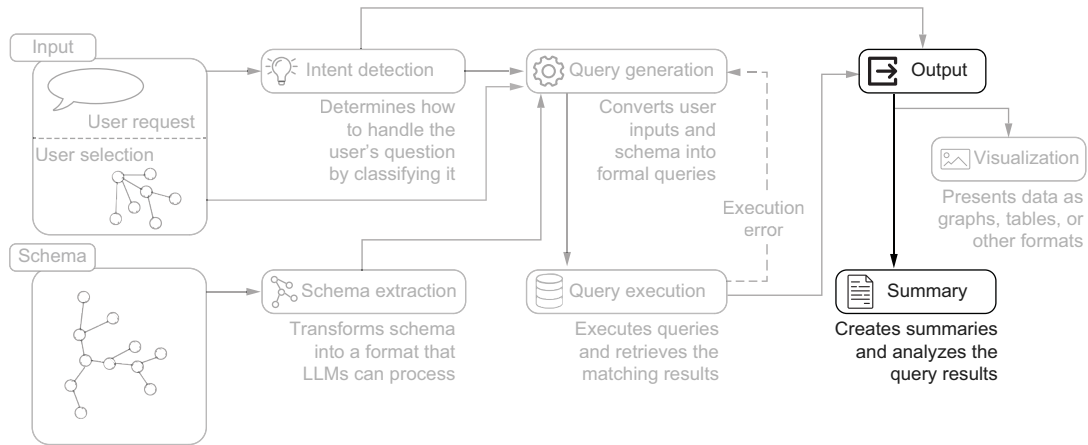


Figure 14.12 Output generation pipeline, highlighting the system's final stages of data presentation and analysis. Processed query results are transformed into visual presentations and analytical summaries, demonstrating the dual-output approach of visualization and summarization of query results.

Beyond its basic role of presenting results clearly, the summarization step can potentially be extended through post-processing to provide additional context and insights. This positions it as not just a final formatting step but rather a vital component for enhancing the overall user experience. Here is the prompt structure we'll use for summarization to achieve these goals:

AN Our user asked this question:

```
<QUESTION>
{{ question }}
</QUESTION>
```

To answer the question, we decided to execute this cypher query:

```
<QUERY>
{{ query }}
</QUERY>
```

The query returned a graph containing this data:

```
<RESULTS>
{{ records }}
</RESULTS>
```

```
{%if selection -%}
```

Current selection:

```
{% for node in selection%}
- {{node.label}} node with this properties {{node.properties}}
{% endfor %}
{%-endif-%}
```

Your task is to summarize the results we sent to the user with the information just provided. Consider that the user will see the results in a graph format within a graphical user interface, but we also want to provide a textual summary along with the canvas.

Please keep in mind that much of the resulting data is actually irrelevant considering the question, but is returned anyway for completeness. Your job is to filter out this data so the summary contains only factual information that is relevant considering the question.

Does the question request analysis of the returned data? If so, include a few sentences to extract the requested analysis/insight.

This is the question again

```
<QUESTION>
{{ question }}
</QUESTION>
```

Answer only in valid JSON in the following JSON format, nothing else (no <ANSWER> tags or code blocks and so on):

```
{
  "results_analysis": true|false, Check if the question contains an implicit or explicit
  request for analysis of the returned raw data
  "reasoning": "...", Scratch pad for your reasoning. include reasoning about the sum-
  mary and reasoning about the result analysis if needed
  "summary": "..." must be a string and a meaningful and factual summary (use \n
  and basic markdown tags to highlight the important bits).
}
```

Let's break down how this prompt is structured to achieve our summarization goals while maintaining flexibility for future enhancements. It begins by reconstructing the complete context chain, from initial question through query execution to results. Each element is carefully wrapped in HTML-like tags, creating clear boundaries that help the LLM understand the role of each component:

- The original question provides the user's intent.
- The executed Cypher query shows how we interpreted that intent.
- The query results represent our raw data.
- The current selection (if any) maintains user context.

The task description explicitly acknowledges the dual nature of our results presentation: users will see both a visual graph representation and our textual summary. This is vital because it guides the LLM to focus on complementing rather than merely repeating what's visible in the graph interface.

We explicitly instruct the LLM to filter out irrelevant data; this is a critical step, given that graph queries often return complete paths for visualization purposes. This filtering ensures that our summaries remain focused and meaningful to the user's original question.

The prompt introduces result analysis capabilities through a simple question: "Does the question request analysis?" This approach keeps the analysis tied to user intent rather than generating unrequested insights, and provides a foundation for future expansion of analytical capabilities while maintaining a clear separation between summary and analysis functions.

Before specifying the output format, we include a reminder of the original question. This repetition, like our approach in previous prompts, ensures that the model maintains focus despite potentially lengthy results sections.

The JSON output format follows our established pattern of progressive generation. First we generate a flag to identify whether the results analysis is required by the user, even if only implicitly. This flag is useful for monitoring purposes, as it compels the model to make an explicit decision about whether to perform the analysis. Additionally, when the flag is `true`, it significantly influences the subsequent reasoning step.

Next, the reasoning step must align with both the prompt and the analysis decision, incorporating any relevant analysis-related reasoning. Finally, we request the generation of the actual summary, including indications for typographic formatting to highlight the content when necessary.

This structured approach ensures that our summaries are consistent, relevant, and adaptable to future enhancements. The clear separation of concerns from decision-making through reasoning to final output makes the component both maintainable and extensible.

The summarization step completes our pipeline by transforming raw data into accessible insights while maintaining the flexibility to grow with evolving user needs. Its position as the final touchpoint with our users makes it a critical component for ensuring that the system answers questions accurately and also presents those answers in the most useful way possible.

Summary

- Expert emulation provides a systematic framework for building, improving, and extending KG systems. When facing any challenge, we can find solutions by asking "What would an expert do?" and breaking down their approach into implementable steps.
- A well-structured intent detection system requires two layers of classification. The first layer handles broader query categories (data-related, system-related, feedback), and the second identifies visualization needs.

- Converting technical database schemas into LLM-friendly formats requires filtering out unnecessary elements, adding contextual annotations, and structuring information in ways that align with how LLMs process and understand data.
- Prompt engineering for LLMs requires giving them “time to think.” This means structuring prompts to encourage reasoning before answering and using techniques like chain-of-thought prompting to improve response quality and reliability.
- To work effectively, query-generation prompts need comprehensive schema context, the current user selection state, intent-specific requirements, and carefully chosen examples that demonstrate desired patterns.
- Result summarization works best as a complement to visualization. Rather than repeating what’s visible in the graph, effective summaries highlight insights and patterns that might not be immediately apparent visually.

15

Building a QA agent with LangGraph

This chapter covers

- Implementing the expert-emulating approach
- Implementing investigation through question answering
- Adapting and improving the system

In this chapter, we'll create a practical application for querying knowledge graphs using LLMs. Drawing together the concepts and techniques explored in chapter 14—illustrated in the mental model in figure 15.1—we'll demonstrate how to build an integrated solution. Using LangGraph as our orchestration framework, we'll show how each stage can be combined into a seamless pipeline. To make this system accessible and user-friendly, we will use Streamlit as a frontend interface. The book's code repository contains the complete implementation and configuration files, so you can easily follow along and reference the code as we progress through the concepts.

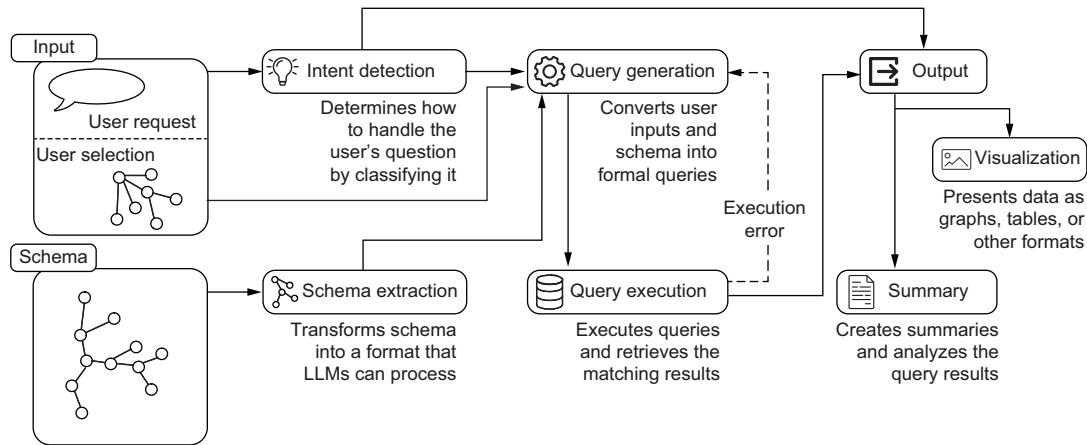


Figure 15.1 Overview of the system architecture introduced in chapter 14. We'll implement it using Streamlit to handle user input (questions and user selection) and output (visualization and summaries), and LangGraph will orchestrate the core pipeline.

15.1 Building the LangGraph pipeline

LangGraph is an innovative library designed for building stateful, multi-actor applications powered by LLMs. This framework is particularly suited for orchestrating workflows that involve complex reasoning and decision-making processes, which are central to our KG querying pipeline.

To better understand how *LangGraph* implements these concepts in practice, let's start with a familiar example: a basic retrieval-augmented generation (RAG) system that retrieves relevant documents and generates an answer to a question. Although simpler than our expert-emulating architecture, this example demonstrates the core principles we'll build on.

As shown in figure 15.2, the workflow consists of two main operations: document retrieval and answer generation. What makes *LangGraph* distinctive is its approach to component communication: rather than passing data directly between components, each one interacts with a shared state, similar to a whiteboard where each agent can read previous work and add their results.

The state begins with the user's question. The first agent, responsible for document retrieval, reads the question from the state and adds the relevant documents it finds. The second agent can then access both the original question and the retrieved documents to generate an appropriate answer, which is in turn added to the state.

At its architectural core, *LangGraph* implements these workflows as directed graphs, where each node represents a distinct agent function. These agent functions carry out their responsibilities by interacting with the global state—reading the relevant data and updating it with their results after execution. The edges in the graph

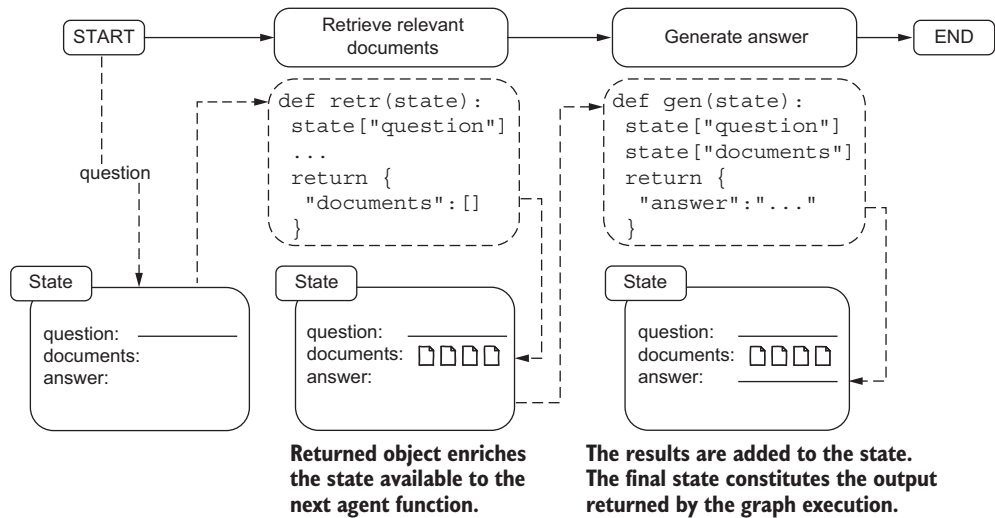


Figure 15.2 State-based communication between agent functions in LangGraph. Agents remain decoupled while communicating through an evolving state object. Each agent function receives and updates the global state independently.

determine the execution flow, specifying which node should be executed next. Importantly, LangGraph supports dynamic edge resolution, enabling workflows to branch based on arbitrarily complex logic. For instance, the next node to execute can be chosen based on the output of a preceding agent, ensuring flexibility and adaptability in designing workflows.

This approach ensures that the system maintains a coherent state throughout the workflow while enabling AI system designers to create a spectrum of applications, from router systems where LLMs are just one component among many, to fully autonomous systems where LLMs can determine and shape their execution paths. This flexibility in flow control makes LangGraph ideal for implementing our expert-emulating architecture, where we need to coordinate multiple specialized phases such as intent detection, schema extraction, and query generation.

15.1.1 System architecture overview

With LangGraph's capabilities clear, let's examine how the expert-emulating approach translates into an executable workflow. Figure 15.3 shows the core components of our KG querying system, from initial user input processing through intent detection, query generation, and result presentation, with each node representing a discrete agent function. Notably, this component structure maps naturally to LangGraph's agent/state architecture, where each processing step can be implemented as an agent function that receives and updates the workflow state.

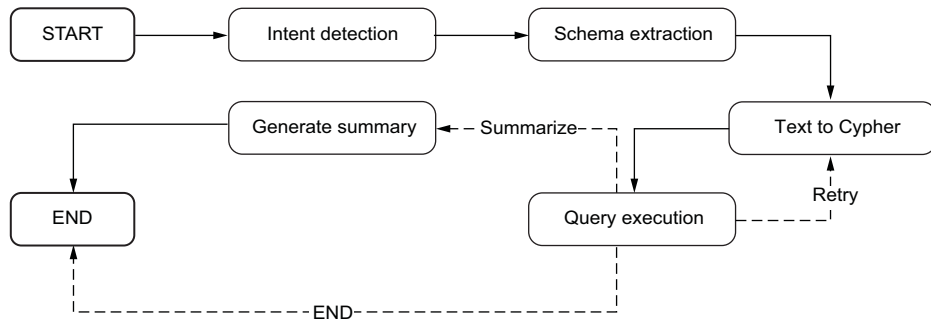


Figure 15.3 LangGraph implementation of the KG querying pipeline. The solid arrows show the main flow from intent detection through schema extraction and query execution, and dashed arrows indicate conditional paths based on query execution outcomes. This directed graph structure directly maps each component of our expert-emulating approach to a LangGraph agent function.

The LangGraph workflow, although central to our backend system, integrates with several supporting components and interfaces with frontend applications; figure 15.4 shows a broader architectural view. At the core of the backend sits our LangGraph workflow, consuming prompts and configuration from the configuration provider while dynamically accessing graph schema information through the schema provider. These supporting components handle critical setup tasks: the configuration provider manages prompt templates and system settings, and the schema provider extracts and formats database schema for LLM consumption.

The question processing interface acts as a bridge between the core pipeline and frontend applications. It exposes the LangGraph workflow as an event stream, allowing frontends to track pipeline progress in real time. This interface processes incoming questions, feeds them through the workflow, and streams state updates and final responses back to the user interface.

This architecture cleanly separates concerns while maintaining the flexibility needed for a conversational AI system. Each component has a clear responsibility: LangGraph handles the core question-answering logic, providers manage configuration and schema access, and the processing interface handles frontend communication.

In the following sections, we'll examine each of these architectural elements, starting with the supporting components that provide configuration management and schema translation services. We'll then explore the state management design that enables effective communication between pipeline stages, followed by the implementation of individual pipeline agents which form the core of our question-answering system. Finally, we'll look at how the pipeline integration layer brings these elements together into a cohesive system that can interact with frontend applications.

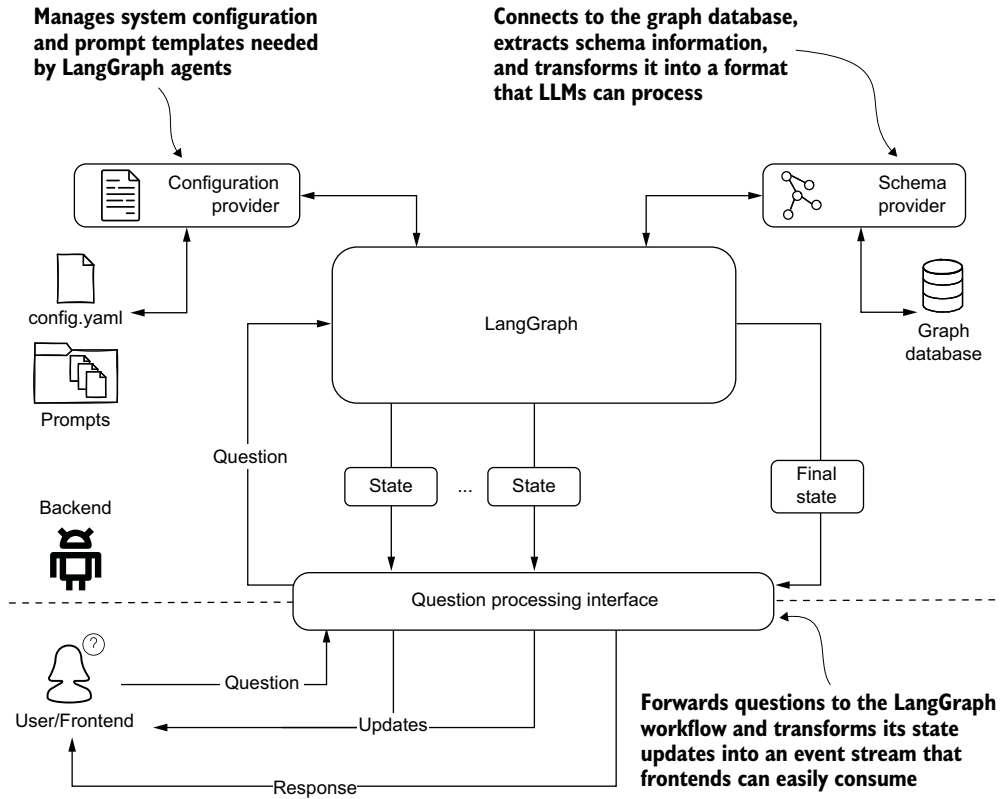


Figure 15.4 Backend architecture showing how the LangGraph pipeline integrates with supporting components. The configuration provider manages prompts and settings, and the schema provider handles database schema access. The question-processing interface bridges the core pipeline with frontend applications through an event-based API.

15.1.2 Configuring pipeline components

Before diving into the configuration details, let's examine how the configuration component fits into our system architecture (see figure 15.5). The configuration component serves as a centralized repository for storing textual elements that our question-answering system relies on, primarily prompt templates and KG annotations. This separation of concerns helps maintain clean and maintainable code by preventing these often lengthy text elements from cluttering the main implementation.

Our templates use the Jinja2 templating language, allowing for dynamic content generation at runtime. By isolating template definitions in the configuration, we create a clear boundary between template authoring and template rendering. The main code can then interact with these templates through a clean interface, remaining unaware of the underlying template composition process.

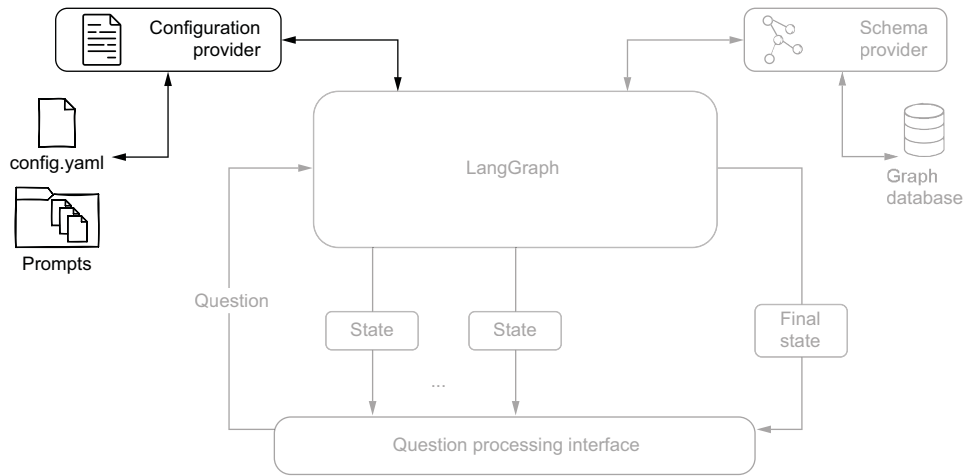


Figure 15.5 System architecture diagram highlighting the configuration provider component. The provider manages system configuration and prompt templates needed by LangGraph agents to process user questions.

This approach offers several advantages. First, it simplifies the tuning of KG descriptions and prompt templates, as all adjustments can be made in a single location without touching the core logic. Second, it provides a structured foundation for future extensions: adding new processing steps or modifying existing ones becomes more straightforward when all text-based resources are organized in one place. Finally, having a dedicated configuration component makes it easier to manage different versions of prompts and annotations, which is particularly valuable during the development and refinement of the system. The following listing shows an example of our configuration structure.

Listing 15.1 Configuration file example

```
notes: >
- all POINTS properties are Neo4j Points (`point.distance()`
  and similar functions work for them)
- do not expand ANPRCameraEvent unless you need
  to connect it to both Vehicle and ANPRCamera
- a previous offender or known offender is defined by the fact that
  the node is connected to crimes

examples:
- question: Crimes that occurred on March 14th, 2025
  answer: MATCH (c:Crime) WHERE c.date starts with "2025-03-14"
  reasoning: >-
    To find the crimes that occurred on that date, we leverage
    the <b>date</b> property of the crime node.
    Since it is formatted as an ISO string, we can use the
    prefix "2025-03-14" to get all crimes that occurred on that day.
    Since there is no traversal, no paths are returned
```

```
[...]
- question: Return one male known offender aged 20 to 22
  answer: >-
    MATCH path = (person:Person)
                -[committed:COMMITTED]->(crime:Crime)
    WHERE (person.sex CONTAINS 'MALE' AND
           person.age >= 20 AND person.age <= 22)
    RETURN path LIMIT 1

prompts:
  text_to_cypher:
    system: >-
      Your task is to generate a Cypher query for a Neo4j graph
      database, based on the schema definition provided,
      that answers the user question.
    template: templates/text_to_cypher.template
  intent_detection:
    template: templates/intent_detection.template
  generate_summary:
    template: templates/summary.template
```

The configuration combines three key elements: operational notes that guide query generation, examples that demonstrate proper query construction, and prompt template references. The notes capture essential domain knowledge and best practices for working with the graph database, and the examples provide question–answer pairs with detailed reasoning that help the LLM understand the expected query patterns. This combination of declarative knowledge and practical examples creates a rich context that guides the LLM in generating accurate queries.

The prompts section references external templates for different pipeline stages (intent detection, query generation, and summary generation), keeping the actual prompt templates separate from their configuration. This separation allows for easier maintenance and version control of both the configuration structure and the prompt content.

Each template can be modified independently without affecting the overall configuration, making it easier to experiment with different prompt variations during development. The next listing shows how the configuration component manages template loading and dynamic content generation.

Listing 15.2 Configuration component

```
class ChainConfiguration:
    def __init__(self):
        self.base = Path(__file__).parent
        self.config = self.load()

    def load(self):
        config_file = self.base / "chain_config.yaml"
        return yaml.load(config_file.open(), Loader=yaml.FullLoader)

    def get_prompt(self, name, **kwargs):
        system = self.config["prompts"][name].get("system")
```



```

template_file = (
    self.base / self.config["prompts"][name]["template"])

template = template_file.read_text()
prompt = jinja2_formatter(template, **kwargs)

return system, prompt

def getAnnotations(self, reload=True):
    if reload:
        self.config = self.load()
    return {
        "notes": self.config["notes"],
        "examples": self.config["examples"]}

```

The `ChainConfiguration` class provides a clean interface for accessing our configuration elements. It offers two main methods: `get_prompt` for retrieving and formatting prompt templates, and `getAnnotations` for accessing the notes and examples. This implementation ensures that all configuration elements are easily accessible while maintaining a clear separation between configuration storage and use.

15.1.3 Schema translation service

Next, let's examine the Schema provider component and its interaction with the graph database in our system architecture (see figure 15.6). The schema provider is a vital component in our expert-emulating question-answering system. Although our goal is to automate schema extraction, we face a fundamental challenge that we explored in chapter 14: we need access to the conceptual schema, but we can only programmatically access the technical schema.

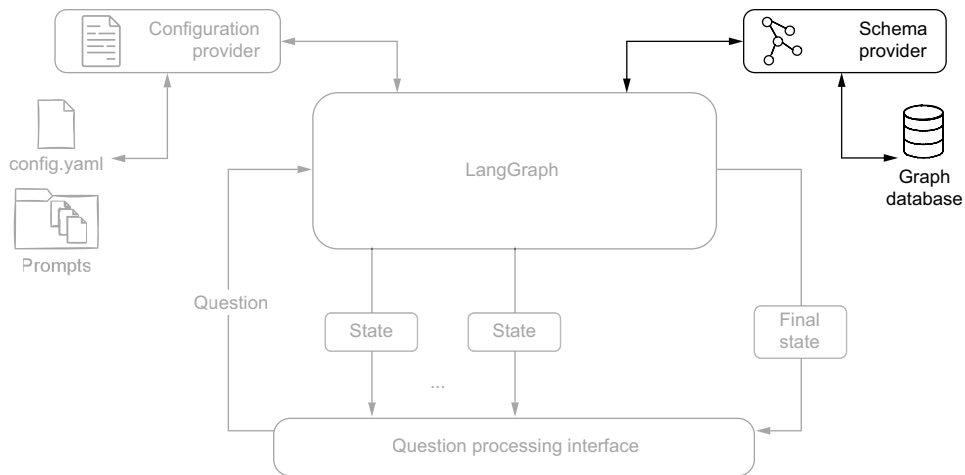


Figure 15.6 System architecture diagram emphasizing the schema provider component, which connects to the graph database to extract and transform technical schema information into LLM-friendly formats

To solve this, we've developed a configuration-based transformation approach that consists of two key components. The first component is a skip list that identifies which elements should be excluded from the conceptual model. These elements include technical nodes and relationships that don't represent business concepts, implementation-specific properties such as internal IDs and timestamps, and elements that would add unnecessary complexity to LLM prompts. The second component is a description section that enriches the filtered schema by adding business-level descriptions for nodes and relationships, contextual information about properties, and domain-specific terminology and constraints.

We store this configuration in YAML format, making it easy to maintain and update as the data model evolves. The schema provider follows a three-step process to transform the raw technical schema into an LLM-friendly format: it extracts the technical schema using Neo4j's `apoc.meta.schema`, filters out technical elements using the skip list, and enriches the remaining elements with business descriptions.

Let's examine how this is implemented in practice. First we need a data model that can represent both technical and enriched schema information, as in the following listing.

Listing 15.3 Schema provider: data model

```
@dataclass
class Property:
    """Represents a node or relationship property with an optional
       description"""
    name: str
    type: str
    description: str = None

    def __str__(self):
        """Represents the property as string in the format:
           property_name:TYPE /* optional description */ """
        ret = f"{self.name}: {self.type}"
        if self.description is not None:
            ret += f" /* {self.description} */"
        return ret
```

← Dataclass representing a node
or relationship property


```
@dataclass
class Node:
    """Represents a node type."""
    items = {}
    name: str
    properties: list[Property]
    description: str = None

    @classmethod
    def mk_node(cls, name, value):
        """Creates a new node with the given name and properties
           from a dictionary.

           Args:
               name (str): The name of the node.
```

← Dataclass representing
a node type

← Keeps track of all
nodes at a global level

← Instantiates nodes using
the node description from
`apoc.meta.schema`

```

        value (dict): the node description as
                        returned by `apoc.meta.schema`
    """
    properties = [Property(name=k, type=v["type"])
                  for k, v in value["properties"].items()]
    properties = sorted(properties, key=lambda x: x.name)

    node = Node(name=name,
                properties=properties)

    for rel_name, rel_value in value["relationships"].items():
        Relationship.mk_rels(source=name, name=rel_name,
                            value=rel_value)

    cls.items[node.name] = node
def drop_properties(self, skipProperties):
    """Drops specified properties from the node.

    Args:
        skipProperties (list): A list of property names
                               to be dropped.
    """
    self.properties = [prop for prop in self.properties
                       if prop.name not in skipProperties]
def __str__(self):
    """Represents the node as string in the format:
       (:NodeType /* node class description */ {
         property_one:TYPE /* property one description */,
         property_two:TYPE /* property two description */,
         ...
       })
    """
    descr = (" if self.description is None
             else f"/* {self.description} */ ")

    return (
        f"({self.name} {descr}{{\n    " +
        ",\n    ".join(str(prop) for prop in self.properties) +
        "\n})\n"
    )

```

← Stores the newly created node instance in the Node.items dictionary

← Recomputes node properties by filtering out properties in the skip list

← Assembles node components to form the desired node description format

This data model provides the foundation for our schema transformation process. The Property class handles individual attributes with their descriptions, and the Node class manages the overall structure and filtering capabilities. Building on this data model, the following listing shows how the Neo4jSchema class implements our core schema management functionality.

Listing 15.4 Schema provider main class

```

class Neo4jSchema:
    [...]
    def get_schema(self):
        with self.driver.session() as session:

```

← Initializes the schema using a technical schema from apoc.meta.schema

```

result = list(session.run(
    "CALL apoc.meta.schema({sample:-1})"
))[0]["value"]

[Node.mk_node(k, v) for k, v in result.items()
 if v["type"] == "node"]

@staticmethod
def apply_configuration(config: dict = None):
    if config is None:
        config_file = Path(__file__).parent / "schema_config.yaml"
        config = yaml.load(config_file.open(),
                           Loader=yaml.FullLoader) ["schema"]

    items = {node.name: node for node in Node.items.values()
             if node.name not in config["skip"]["classes"]}
    Node.items = items

    for node in Node.items.values():
        node.drop_properties(config["skip"]["properties"])

    for node in Node.items.values():
        node.description = (config["descriptions"]["classes"]
                           .get(node.name))

    for prop in node.properties:
        property_description = (config["descriptions"]["properties"]
                               .get(node.name, {})
                               .get(prop.name))
        prop.description = property_description

    skip = config["skip"]
    relationships = {rel_name: rel
                     for rel_name, rel in Relationship.items.items()
                     if rel.source not in skip["classes"]
                     if rel.dest not in skip["classes"]
                     if rel.name not in skip["relationships"]}

    for rel in Relationship.items.values():
        rel.drop_properties(config["skip"]["properties"])

    Relationship.items = relationships

def __str__(self):
    ret = ["### Graph Schema Overview\n",
          "#### Node Types"]
    ret += [str(node) for node in Node.items.values()]
    ret.append("#### Relationships\n")
    ret += [str(rel) for rel in Relationship.items.values()]
    return "\n".join(ret)

```

Parses apoc.meta.schema results using list comprehension →

Calls apoc.meta.schema on the full database without sampling ←

Converts the technical schema to conceptual by applying filters and descriptions ←

Uses schema_config.yaml from the package directory if no configuration is provided →

Recomputes the node types, filtering out skip-list nodes ←

Removes properties in the skip list from Node objects →

Uses the description from descriptions.classes.<class_name> if exists →

Looks for descriptions.properties.<class_name>.<property_name> ←

Filters out relationships with the source node in the skip list →

Filters out relationships with the destination node in the skip list →

Filters out relationships in the skip list ←

Removes properties in the skip list from Relationship objects ←

Creates a Markdown representation of the schema with node types and relationships ←

The `get_schema` method retrieves the technical schema, and `apply_configuration` handles the transformation process according to our configuration. This implementation ensures that LLMs receive a clean, conceptual view of our data model while maintaining all necessary information for query generation.

In practice, this transformed schema serves several important purposes for our LLM components. It enables them to understand the domain model at a conceptual level, generate queries using proper entity and relationship names, and consider business rules and constraints during query construction. This approach creates an effective balance between maintaining technical accuracy and providing LLMs with an accessible, business-oriented view of our data model.

15.1.4 State management design

The cornerstone of agent communication in LangGraph is the `state` object, which serves as a shared memory space that agents can read from and write to. Each agent is responsible for populating specific portions of this state, creating a clear chain of responsibility throughout the pipeline. Let's examine the structure of this `state` object.

Listing 15.5 Pipeline agent's state

```
class AgentState(TypedDict):
    question: str
    output_type: str
    output_type_reason: str
    schema: str
    query: str
    query_reasoning: str
    query_message: str
    results_error: list
    summary: str
    summary_reason: str
    summary_analysis: bool
    information: str
    retries: int
```

This state structure can be broken down into logical sections:

- *Question input* (`question`)—Stores the original user request
- *Intent detection results* (`output_type`, `output_type_reason`)—Captures the detected visualization intent and the reasoning behind it
- *Schema information* (`schema`)—Contains the graph schema in LLM-friendly format
- *Query generation* (`query`, `query_reasoning`, `query_message`)—Holds the generated Cypher query and associated metadata
- *Error handling* (`results_error`)—Tracks any errors encountered during query execution
- *Summary generation* (`summary`, `summary_reason`, `summary_analysis`)—Contains the generated summary and analysis flags
- *Queries retry mechanism* (`information`, `retries`)—Manages the retry logic for failed queries

Each field tells a story about the pipeline's progress. The state not only carries data between agents but also maintains the context necessary for making routing decisions and handling errors gracefully.

15.1.5 Pipeline agent implementation

As we discussed earlier, our expert-emulating approach maps naturally to a LangGraph pipeline, where each step in our question-answering process is implemented as a specialized agent. Figure 15.7 shows this pipeline structure again, highlighting how agents are connected and how the flow of information progresses from the initial question to the final answer. In this section, we will look at how these agents are made and how they interact with the LangGraph state object.

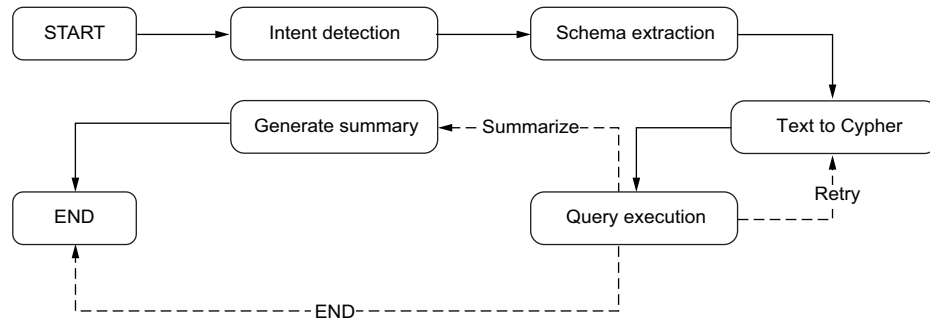


Figure 15.7 LangGraph implementation of the knowledge expert-emulating graph-querying pipeline

INTENT DETECTION AGENT

The intent detection agent serves as the entry point of our pipeline: it determines how the user's question should be visualized. It operates solely on the user's input question and enriches the state with visualization intent information.

This agent uses the intent detection prompt we discussed in chapter 14 to analyze the question and determine the most appropriate visualization type. The agent updates two key fields in the state:

- `output_type`—The determined visualization format (table, graph, or map)
- `output_reason`—The reasoning behind the selected visualization type

Let's examine the implementation in listing 15.6.

Listing 15.6 Intent detection agent implementation

```

def run_prompt(self, prompt, system=""):
    messages = [HumanMessage(content=prompt)]
    if self.system or system:
        system = self.system if not system else system
        messages = [SystemMessage(content=system)] + messages

    message = self.model.invoke(messages)
  
```

Handles prompt execution and response processing

Prepends a system message to the prompt if provided

```

logger.debug(f" got {message.content}")
payload = message.content
payload = re.sub(r'^\s*````json\s*|\s*````\s*$', '',
                payload, flags=re.DOTALL)
return json5.loads(payload)

def intent_detection(self, state: AgentState):
    system, prompt = self.config.get_prompt(
        "intent_detection", question=state["question"])
    results = self.run_prompt(prompt, system)
    return {
        "output_type": results["type"],
        "output_reason": results["reason"]}

```

Removes JSON code block markers from the response if present

Parses the response as JSON using the more lenient JSON5

Retrieves and renders the intent detection prompt template from configuration

Maps the response fields to their corresponding state properties

The agent retrieves the intent detection prompt template from the configuration, executes it with the user's question, and processes the LLM's response. The response is expected in a JSON format containing the visualization type and the reasoning, which are then mapped to the corresponding state fields.

SCHEMA EXTRACTION AGENT

The schema extraction agent serves as a bridge between our KG and the LLM components by using the Neo4jSchema object we introduced in section 15.1.3. Its primary responsibility is to convert the KG schema into an LLM-friendly format that subsequent agents can use for query generation and reasoning. The next listing shows how the agent handles prompt execution and response processing, with particular attention to JSON parsing and error handling.

Listing 15.7 Schema extraction agent

```

def schema_extraction(self, state: AgentState):
    assert self.neo4j_schema is not None, \
        "you need to provide a neo4j schema"

    self.neo4j_schema.get_schema()
    self.neo4j_schema.apply_configuration()
    return {"schema": str(self.neo4j_schema),
            "retries": 0}

```

Retrieves the full technical schema

Filters unnecessary schema components and adds descriptions as per the configuration

Updates the state properties with the LLM-friendly version of the schema and resets the retries counter

The heavy lifting is performed by the Neo4jSchema instance. The agent first ensures that a neo4j_schema object has been provided and then retrieves the current schema and applies any configuration settings we've defined. The resulting schema is converted to a string format that LLMs can effectively process. Additionally, the agent initializes the retry counter to zero, preparing the state for potential query retries later in the pipeline.

TEXT-TO-CYPHER AGENT

The text-to-Cypher agent transforms the user's natural language question into a Cypher query, considering both the graph's schema and any currently selected elements in the visualization. This contextual awareness allows users to reference selected nodes or relationships without explicitly describing them, making queries more natural and concise, as we discussed in section 14.7.2. As shown in the following listing, the agent adds two pieces of information to the state before executing the prompt: annotations from the configuration provider that help guide query generation, and the current selection state from the visualization interface.

Listing 15.8 Text-to-Cypher agent

```
def text_to_cypher(self, state: AgentState):
    extra = {
        "annotations": self.config.get_annotations(),
        "selection": self.selection
    }
    params = dict(state) | extra

    system, prompt = self.config.get_prompt("text_to_cypher", **params)
    logger.debug(f"prompt: {prompt}")

    results = self.run_prompt(prompt, system)

    return {"query": results["query"],
            "query_reasoning": results["reasoning"],
            "query_message": json.dumps(results)}
```

The agent merges the current state with the additional context (`extra`), retrieves and populates the appropriate prompt template, and then processes it through the LLM. The results include the generated Cypher query, the reasoning behind its construction, and the complete LLM response for debugging purposes. These are stored in the state's `query`, `query_reasoning`, and `query_message` fields, respectively.

QUERY EXECUTION AGENT

The query execution agent, shown next, provides robust error handling and dynamic result formatting based on visualization needs.

Listing 15.9 Query execution agent

```
def query_execution(self, state: AgentState):
    try:
        results = self.neo4j_schema.run(state["query"])
        if state["output_type"] in {"graph", "map"}:
            self.results = list(results)
        else:
            self.results = results.to_df()
        results_error = None
        information = ""
    except neo4j.exceptions.ClientError as e:
```



```

self.results = None
results_error = str(e)
logger.info(f"got error: {e}")
information = f"""We tried:
                {state['query']}
                and we got:
                ~~~
                {str(e)}
                ~~~"""

retries = state.get("retries", 0) + 1
return {"results_error": results_error,
        "retries": retries,
        "information": information}

```

The agent's logic is straightforward. It attempts to execute the query stored in the state and processes the results based on the detected intent. When dealing with graph or map visualizations, it preserves the results in their native format as a list of records. However, for tabular output, it converts the results into a pandas `DataFrame` using Neo4j's built-in conversion functionality.

Error handling is a vital aspect of this agent. If the query execution fails (typically due to syntax errors or schema mismatches), the agent performs several operations: it captures the error details, logs the failure for debugging purposes, constructs an error message that includes both the attempted query and the error description, and increments the retry counter to keep track of attempts.

The agent updates the state with three pieces of information. The `results_error` field contains the error message if execution failed, remaining `None` otherwise. The `retries` field tracks the number of execution attempts, and the `information` field provides detailed context about the error for potential retry attempts. This error information is important for the post-execution routing logic, which we'll examine next.

POST-QUERY EXECUTION

Figure 15.8 illustrates the routing logic implemented by this component, highlighted in the context of the overall pipeline. Unlike other components we've discussed, the

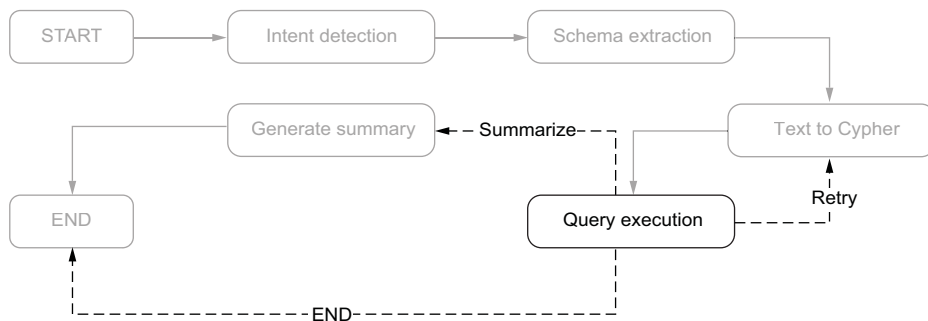


Figure 15.8 Post-query execution routing logic in the QA pipeline, showing decision paths for retry, summarization, and direct completion

post-query execution is not an agent but rather implements the routing logic as a dynamic edge in our LangGraph pipeline (see the following listing).

Listing 15.10 Post-query execution dynamic edge

```
def post_query_execution(self, state: AgentState):
    if state["results_error"] is not None:
        if state["retries"] < 3:
            logger.info(f"{state['retries']} runs, we retry")
            return "retry"
        else:
            logger.info(f"{state['retries']} runs are enough")
            return "END"

    if state["output_type"] in ("map", "graph"):
        logger.info("summarizing..")
        return "summarize"
    else:
        logger.info("no summarization is needed")
        return "END"
```

Handles query execution failures with retry logic up to three attempts

Routes to summarization for map/graph outputs; otherwise, completes

The routing logic follows two decision paths. First, it handles query execution failures by checking the `results_error` field in the state. If an error occurred, the component implements a retry mechanism that allows up to three attempts to execute the query. This gives our system resilience against temporary failures or cases where the LLM may need multiple attempts to generate a correct query.

Second, for successful queries, the routing decision depends on the visualization intent captured in `output_type`. When dealing with map or graph visualizations, the component routes the flow to the summarization step, as these visualization types benefit from additional context and explanation. However, for tabular results, which are typically self-explanatory, the pipeline can conclude directly.

This dynamic routing capability is a key feature of LangGraph, allowing us to implement complex flow control based on both execution results and user intent. The component's simplicity belies its importance in orchestrating the overall pipeline behavior.

GENERATE-SUMMARY AGENT

The final agent in our pipeline generates summaries for graph and map visualizations (see listing 15.11). It combines the query results and the schema selection to create a comprehensive context for the summary generation. It merges these elements with the existing state to populate the parameters for the summary prompt template. The configuration provider supplies the appropriate prompt template, which is then executed through the LLM.

Listing 15.11 Generate-summary agent

```
def generate_summary(self, state: AgentState):
    extra = {
        "records": self.results,
        "selection": self.selection
    }
    params = dict(state) | extra
    system, prompt = self.config.get_prompt(
        "generate_summary", **params)

    logger.debug(prompt)

    results = self.run_prompt(prompt, system)
    return {"summary": results["summary"],
            "summary_reason": results["reasoning"],
            "summary_analysys": results["results_analysis"]}
```

The agent's output enriches the state with three components: the actual summary text, the reasoning behind the summary, and a flag indicating whether additional analysis was performed. This completes our pipeline, transforming the original user question into a meaningful summary of the retrieved information.

PIPELINE ASSEMBLY

The implementation of our expert-emulating pipeline culminates in the construction of the LangGraph workflow. The next listing connects our agents into a cohesive graph structure.

Listing 15.12 Building the LangGraph pipeline graph

```
class Agent:
    def __init__(self, model):
        self.neo4j_schema: Neo4jSchema = None
        self.selection = []
        self.results = None
        self.config = ChainConfiguration()
        graph = StateGraph(AgentState)
        graph.add_node("intent_detection", self.intent_detection)
        graph.add_edge("intent_detection", "schema_extraction")
        graph.add_node("schema_extraction", self.schema_extraction)
        graph.add_edge("schema_extraction", "text_to_cypher")
        graph.add_node("text_to_cypher", self.text_to_cypher)
        graph.add_edge("text_to_cypher", "query_execution")
        graph.add_node("query_execution", self.query_execution)
        graph.add_conditional_edges("query_execution",
                                    self.post_query_execution,
                                    {"retry": "text_to_cypher",
                                     "summarize": "generate_summary",
                                     "END": END})
        graph.add_node("generate_summary", self.generate_summary)
        graph.add_edge("generate_summary", END)
        graph.set_entry_point("intent_detection")
        self.graph = graph.compile(checkpointer=self.memory)
        self.model = model
```

The graph construction follows a clear sequential pattern that mirrors our question-answering workflow. The `StateGraph` class from LangGraph provides the foundation, initialized with our `AgentState` type to ensure type safety throughout the pipeline.

Each agent is added as a node in the graph, with edges defining the standard flow between them. This graph structure implements our expert-emulating approach and provides the flexibility to handle errors and different visualization requirements seamlessly in a single unified pipeline.

15.1.6 Pipeline integration layer

Although LangGraph provides powerful capabilities for building complex workflows, we need to consider how applications will interact with these pipelines in practice. The simplest approach would be to use LangGraph's invoke mode, where we provide an initial state and receive the final result once the pipeline completes. However, this would lead to a less-than-ideal user experience: users might wait for extended periods without any feedback about what's happening behind the scenes. Figure 15.9 illustrates the integration architecture that enables real-time interaction between the LangGraph pipeline and frontend applications.

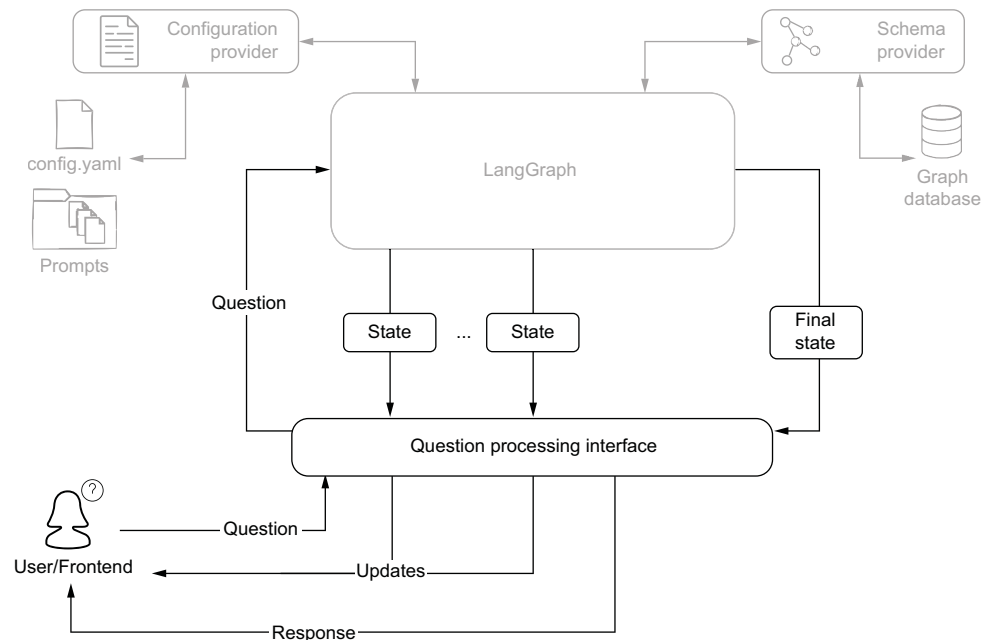


Figure 15.9 Pipeline integration architecture showing the question processing interface mediating between LangGraph state updates and frontend interactions

LangGraph offers a stream execution mode that provides visibility into intermediate steps, but managing these updates directly can complicate the application logic. To

balance the benefits of streaming with ease of use, we've developed an interface layer that transforms the pipeline execution into a series of well-defined events that front-ends can easily consume.

The core of this interface is a *generator function* that processes questions and yields a sequence of events. Generator functions are well-suited for this task because they let us maintain a simple, linear code flow while producing intermediate results. The pipeline execution transforms into a clean event stream featuring strong event typing and comprehensive state tracking, as shown in the following listing.

Listing 15.13 Question-processing interface function

```
def processQuestion(question, selection=None):
    config = { "configurable": {"thread_id": uuid.uuid4().hex}}
    if selection is not None:
        pipeline.selection = [{"labels": list(node.labels)[0],
                                "properties": dict(node)}
                               for node in selection]
    else:
        pipeline.selection = []
    input = {"question": question}
    results = pipeline.graph.stream(input,
                                    config=config,
                                    stream_mode="updates")

    yield "update", "*detecting intent...*", input
    for result in results:
        node, value = list(result.items())[0]
        logger.info(f"got results: {node}, keys: {list(value.keys())}")
        current_state = pipeline.graph.get_state(config).values
        match node:
            case "intent_detection":
                yield "update", "*extracting schema...*", current_state
            case "schema_extraction":
                yield "update", "*generating query...*", current_state
            case "text_to_cypher":
                yield "update", "*executing the query...*", current_state
                yield "result", ("Reasoning", value["query_reasoning"]),
                               current_state
            case "query_execution":
                if value["results_error"]: #J
                    yield "result", ("ERROR", value["results_error"]),
                                   current_state
                else:
                    output_type = current_state["output_type"]
                    yield output_type, pipeline.results, current_state
                    if output_type in {"graph", "map"}:
                        yield "update", "*summary generation...*",
                               current_state
                    # Otherwise, emits a graph/table/map event with the results as payload
```

Configures the pipeline execution with a unique ID

Sets up the internal selection list of dictionaries when a non-empty selection is provided

Processes each pipeline event in this loop until completion

Invokes the pipeline in stream mode, where each update contains only the changed portion of the state

First update: tells the user that the pipeline is running the first step

Extracts the agent's name and state updates from the LangGraph result format

Extracts the current full state

In case of intent detection or schema extraction, notifies the user of the next step

Surfaces the text-to-Cypher reasoning as an intermediate result

In case of an error in the execution of the query, surfaces the error message as an intermediate result

Otherwise, emits a graph/table/map event with the results as payload

```

        case "generate_summary":
            yield "result", ("Summary", value["summary"]),
                           current_state
    logger.info("no more results sendin END")
    current_state = pipeline.graph.get_state(config).values
    yield "END", current_state, current_state

```

When the pipeline completes, fetches the final agent state

Surfaces the summary as an intermediate result

Emits an END event containing the final agent state

The function begins by setting up the initial configuration and state, and then it yields its first update event to inform users that intent detection has started. As the pipeline processes through each node, the generator yields appropriate events to keep the frontend informed. Each case in the pattern-matching structure corresponds to a specific pipeline stage, producing events that align with the current operation.

Each event produced by the generator follows a consistent structure: a triplet containing the response type, the response payload, and the current state of the pipeline. The response types fall into three categories:

- *Update events* inform users about the pipeline’s progress. These events carry simple status messages like “detecting intent” or “generating query,” helping users understand which step is currently executing.
- *Result events* deliver textual outputs such as reasoning steps, potential errors, or generated summaries. These provide deeper insight into the pipeline’s decision-making process and help users understand how the system arrived at its conclusions.
- *Visualization events* represent structured outputs like graphs, maps, charts, or tables. These events carry the data needed to create visual representations of the query results, allowing frontends to present information in the most appropriate format.

By including the current pipeline state with each event, we provide frontends with complete context without making assumptions about how they may use this information. This approach maintains a clean separation of concerns: the interface layer focuses on transforming pipeline execution into a stream of well-defined events while leaving presentation decisions to the frontend.

The result is an interface that keeps users informed and engaged throughout the question-answering process while maintaining clean architectural boundaries. By transforming complex pipeline execution into a simple stream of events, we’ve created a foundation that supports rich interactive experiences without sacrificing maintainability or flexibility.

15.2 Streamlit application

Having built our LangGraph pipeline for expert-emulating question answering, we need an interface that allows users to effectively interact with and validate the system’s capabilities. This interface must support several requirements that emerge from our expert-emulating approach.

The interface needs to support interactive graph visualization, allowing users to explore and select nodes and relationships. It must provide real-time feedback as the pipeline processes questions through its various stages. A chat-like interface is essential for natural language interaction; and the system needs to maintain complex state information about selected graph elements and processing context.

Streamlit's features align well with these requirements. Its native support for chat interfaces provides the foundation for implementing our question-answering interactions, complete with user messages and system responses. The framework's built-in data visualization capabilities, combined with its extensibility through custom components, enable us to create effective graph representations. Most importantly, Streamlit's Python-first approach ensures seamless integration with our LangGraph pipeline: there's no need to build complex APIs or handle cross-language serialization, as both frontend and backend operate in the same Python environment.

As our pipeline processes questions, it generates updates about its progress and intermediate results. Streamlit's session state system, combined with its automatic UI updating, lets us reflect these changes in real time without building event-handling mechanisms. Users can see exactly how the expert-emulating system processes their questions.

These characteristics make Streamlit particularly suitable for prototyping and testing our system. The rapid iteration cycle means we can quickly validate how different types of questions are processed and how various visualization options perform. The framework's low implementation overhead lets us focus on testing and refining the core expert-emulating functionality rather than dealing with frontend complexity. Although a production deployment may warrant a more specialized interface, Streamlit provides exactly what we need for developing and demonstrating our system's capabilities.

15.2.1 *Application overview*

Our next step is designing a functional interface that fully supports expert-emulating graph exploration. The interface must transform abstract requirements like “natural interaction” and “real-time feedback” into components that work together seamlessly. Figure 15.10 shows the main interface layout of our application.

The interface components map directly to the capabilities of our expert-emulating pipeline. Each element is designed to support specific aspects of the system.

At the heart of the application is the graph canvas, which visualizes the KG's nodes and relationships. This central view is complemented by different interaction areas that support natural exploration and questioning workflows.

The Selection column makes questions more natural and context-aware. Users can select specific nodes from the graph and then refer to these selections in their questions using natural language. For example, with certain nodes selected, a user may ask “What are the companies related to these assets?” rather than having to specify each asset explicitly. This selection mechanism is important for testing our system's context-awareness and verifying how well the pipeline understands and incorporates visual context into its query generation process.

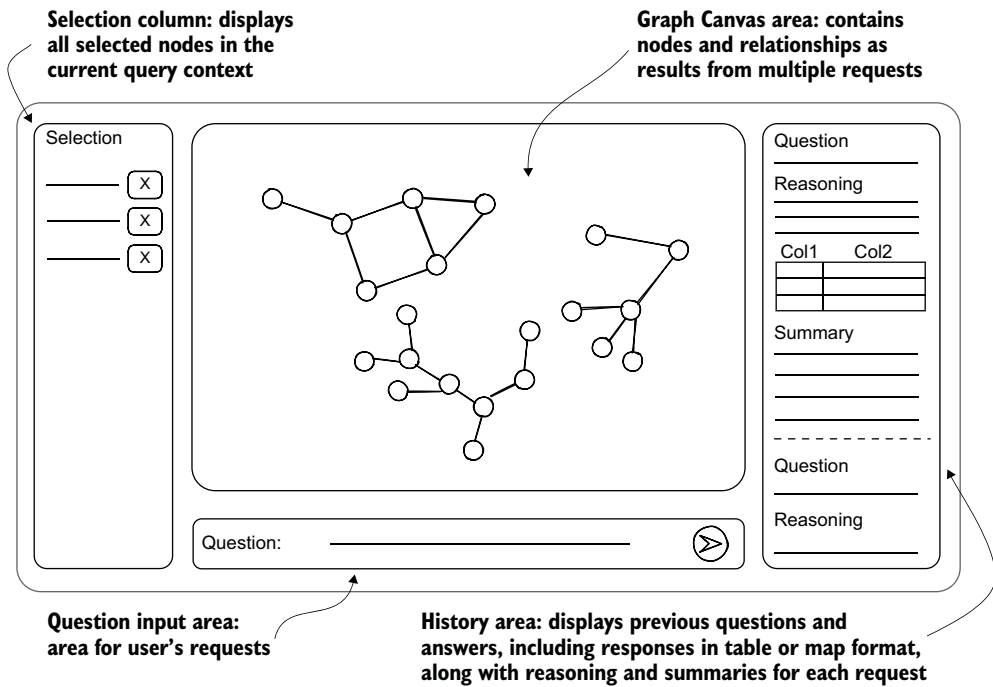


Figure 15.10 Application interface layout demonstrating a question-answering system with selection capabilities, interactive graph visualization, and real-time response tracking

The question input area at the bottom enables users to pose their questions in natural language. These questions can range from simple fact-finding to complex relationship analysis, all while maintaining a natural conversation-like interaction style.

The history area on the right provides a comprehensive view of the question-answering process. Our expert-emulating approach can generate different types of responses, so this area adapts to display various formats. When answers include geographic information, they are presented as interactive maps. When the system determines that tabular data will be most informative, it displays well-formatted tables. Importantly, this area updates in real time as the system processes each question, showing intermediate steps and final results as they become available through our pipeline.

The history area's real-time updates serve a dual purpose: they keep users informed of progress while also making the expert-emulating pipeline's reasoning process visible. This transparency helps users understand how their questions are being processed and why particular visualizations or response formats were chosen.

This design creates a fluid experience where users can explore the KG, ask questions about what they see, and receive rich, multiformat responses that suit the information being conveyed. Although each question stands as an independent interaction, the continuous updates and preserved history create a seamless exploration experience.

15.2.2 LangGraph integration

The integration between Streamlit and our LangGraph pipeline creates a real-time, interactive experience for users. Let's explore how these systems work together.

The integration follows an event-driven pattern. As soon as the user clicks the Send button, the question travels through the question processing interface to the LangGraph pipeline. Rather than waiting for the final result, our system provides immediate feedback as each agent in the pipeline processes the question. This real-time visibility helps users understand how their questions are being analyzed and answered.

As shown in the following listing, to manage this flow of information we implement a `MessageHistory` object that serves two purposes. First, it maintains the complete history of interactions, allowing users to review past questions and answers. Second, it stores the current state of the pipeline, tracking which agent is actively processing the question and what intermediate results have been produced.

Listing 15.14 Message history implementation

```
class MessageHistory:
    def __init__(self):
        self.messages = [{}]

    def update(self, message, finalize=False):
        self.messages[-1].update(message)
        if finalize:
            self.messages.append({})

    @staticmethod
    def display_message(msg):
        with st.chat_message("user"):
            st.markdown(msg["question"])
        with st.chat_message("assistant"):
            if "query_reasoning" in msg:
                st.markdown(f"##### Reasoning\n\n**output type**:\n\n{msg['output_type']}\n\n{msg['query_reasoning']}")
            if "table" in msg:
                st.table(msg["table"])
            if "map" in msg:
                map_ = folium.Map()
                nodes_to_map(msg["map"], map_)
                st_folium(map_)
            if "query" in msg:
                with st.expander("Query...", expanded=False):
                    st.markdown(f"```cypher\n\n{msg['query']}\n\n```")
                    st.json(msg["query_message"])
            if "summary" in msg:
                st.markdown(f"##### Summary\n\n{msg['summary']}")
                with st.expander("extra...", expanded=False):
                    st.json({
                        "summary_reason": msg["summary_reason"],
                        "summary_analysys": msg["summary_analysys"]
                    })
```

Messages are stored as a list of dictionaries; the last one always represents the current message.

The dictionary representing the current message is updated with the new data.

When the message is complete, a new empty dictionary is added to store the next message.

Defines how a single message is displayed in the interface

Converts graph data stored in the "map" key into a format compatible with the map visualization library

Shows the generated Cypher query in a collapsible section along with text-to-Cypher process details

Adds summary generation details in a collapsible section for debugging purposes

The display adapts based on keys in the dictionary that signal which sections need to be rendered.

```

    })
    st.json(msg, expanded=False)
def display_messages(self):
    for message in self.messages:
        if not message:
            continue
        self.display_message(message)

```

← Includes the complete message state for debugging

← Displays messages sequentially

The `MessageHistory` class maintains a list of message dictionaries, with each message potentially containing different types of content—from simple text to complex visualizations. The `update` method allows for progressive building of messages, reflecting the step-by-step nature of our pipeline’s processing. The `display_message` method renders different content types using Streamlit’s components: Markdown for text, tables for structured data, and the Python’s `folium` library for maps. The implementation organizes the information hierarchy, using expandable sections for detailed information like queries and summary reasoning.

The integration between user input and the LangGraph pipeline uses a reactive pattern that manages both temporary and permanent state updates. When processing a question, the system needs to show immediate feedback while also maintaining a permanent conversation history. This is achieved through two complementary mechanisms:

- Event handling with temporary placeholders shows real-time updates as the pipeline processes the question. These updates give immediate feedback but are transient, using Streamlit placeholders to display the current pipeline state.
- `MessageHistory` accumulates the permanent state of the conversation. Rather than showing updates directly, it collects the complete state for each message until it receives an `END` event. Then the page re-renders using `MessageHistory`’s display logic, replacing the temporary updates with the final, persistent version of the conversation.

This approach, implemented in the following listing, ensures that users see both immediate feedback and a permanent record of their interaction history.

Listing 15.15 User input handler

```

[...]
```

```

if question := st.chat_input("What is up?"):
    with chat:
        with st.chat_message("user"):
            st.markdown(question)
        with st.chat_message("assistant"):
            placeholder = st.empty()

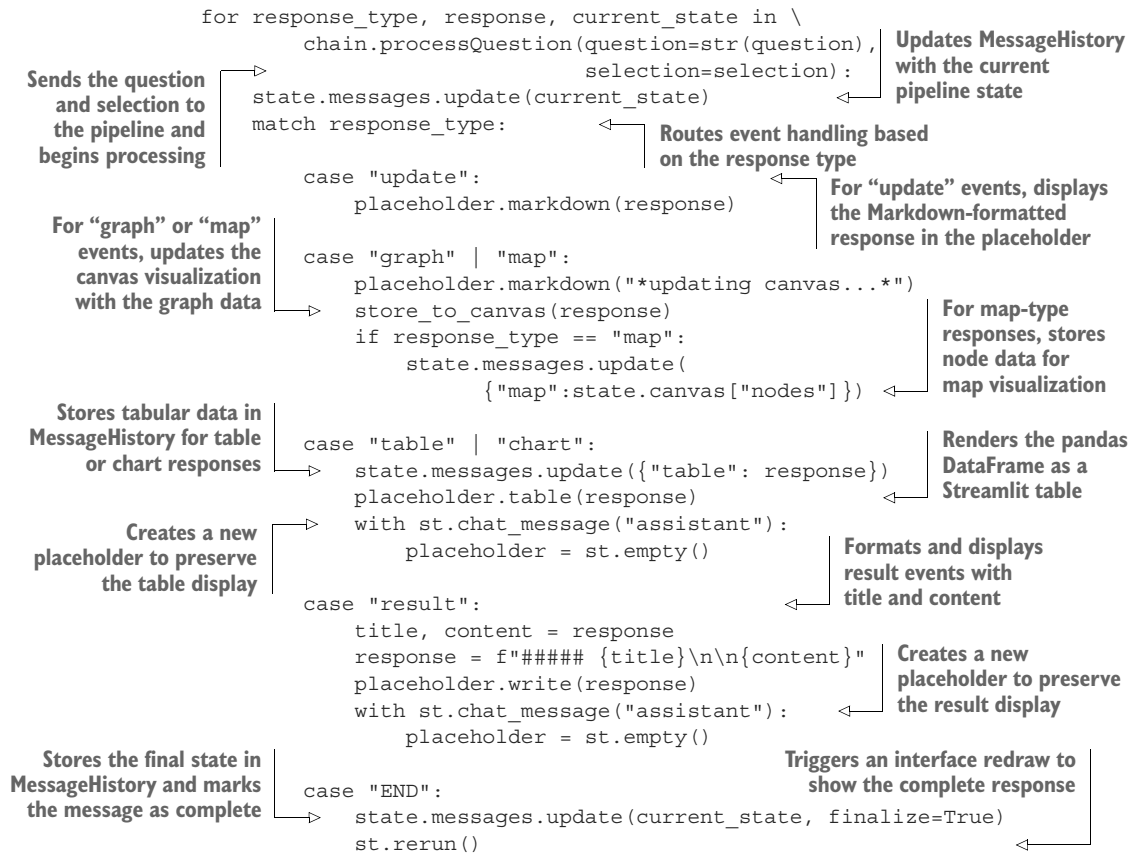
            selection = [state.canvas["byId"][int(item)]
                        for item in state.selection]

```

← Displays the user’s question in the chat history under the “user” role

← Creates a placeholder in the “assistant” section for displaying real-time updates

← Extracts selected nodes from the canvas state using their IDs



When a user submits a question, the system creates placeholder elements in the chat interface that are progressively updated as the pipeline processes the question. The `match` statement handles different types of responses: updates to the chat interface for text responses, graph and map visualizations that are rendered in the canvas, tabular data that is displayed using Streamlit’s table component, formatted results with titles and content, and an `END` event that triggers a rerun to ensure that all UI elements are properly updated. The combination of `MessageHistory` and this event-handling system creates a responsive and interactive experience for users.

15.3 Expert-emulating investigation

To demonstrate how our expert-emulating system works in practice, let’s follow a realistic investigation workflow. We’ll see how an investigator can use natural language queries to explore connections between crimes, surveillance cameras, and vehicles while using the system’s ability to understand context and provide meaningful insights.

Our investigation will work with a subset of the KG focused on criminal incidents. As shown in figure 15.11, the schema connects *Crime* nodes (containing properties

like location, description, and date-time) to *ANPRCamera* nodes (automatic number plate recognition cameras) through spatial relationships. The cameras generate *CameraEvents* when they detect vehicles, recording both the timestamp and location of each sighting. These events link to *Vehicle* nodes, which store properties such as `model`, `color`, and `plate number`. Vehicles in turn connect to *Person* nodes representing their owners, who may have links to *Crime* incidents through `COMMITTED` relationships.

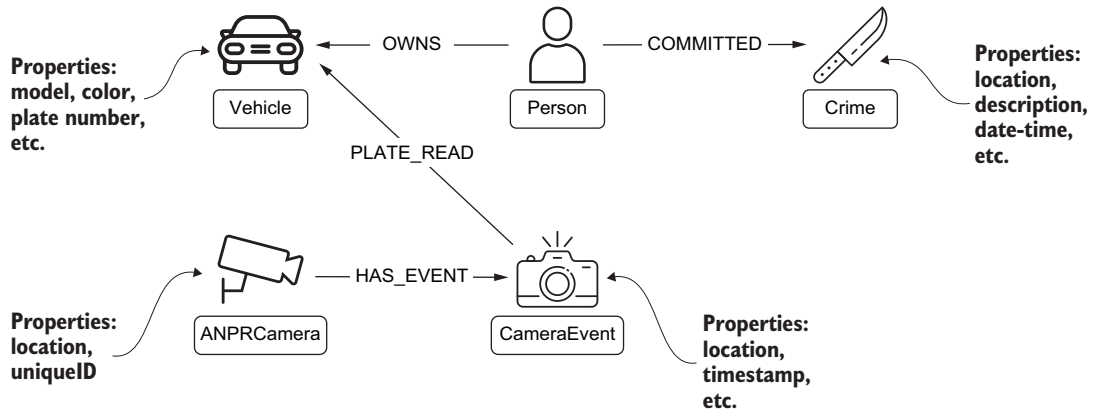


Figure 15.11 Focused schema visualization showing how *Crime*, *ANPRCamera*, *CameraEvent*, *Vehicle*, and *Person* nodes interconnect for investigative queries

This seemingly simple structure allows for sophisticated queries that combine spatial analysis, temporal patterns, and relationship exploration. Let’s see how an investigator can use our system to identify vehicles of interest related to a criminal trespass incident.

15.3.1 Identifying the initial case

Our investigation begins by asking the system to identify a crime currently under investigation. This demonstrates the ability to understand and translate natural language queries that combine both explicit constraints (“currently under investigation”) and implicit expectations (returning a meaningful crime for analysis). The system processes this request through our expert-emulating pipeline, first detecting that a graph visualization will be most appropriate for displaying a single node with its properties. The query generation component understands that we’re looking for an active investigation and includes relevant constraints in the generated Cypher query.

Let’s pose our first question to the system: “Return one crime node currently under investigation”. Figure 15.12 shows the system’s response, displaying a crime node representing a criminal trespass incident.

(a)

Selection

clear selection

Crime (4383354) { ... } remove

Canvas

Crime

```

date
2023-06-15T16:48:00Z
updated_on
06/15/2023 05:07:00 PM
primary_type
CRIMINAL TRESPASS
__hume__createdViaHume
true
description
The victim reported observing
a BLACK vehicle with license
plates beginning with "EB"
leaving the area shortly after
the incident occurred.
id
4383354
position
-87.6195518,41.695476192
uuid
6e05ea0e-159b-45d6-b87a-
b8c894798fd9
status
investigation

```



CRIMINAL TRESPASS

(b)

Chat

Return one crime node currently under investigation

Reasoning

output type: `graph`

To find a crime node currently under investigation, we need to filter the Crime nodes by the status property with the value 'investigation'. Since we only need one such node, we can use LIMIT 1.

Query...

```

MATCH (c:Crime) WHERE c.status =
'investigation' RETURN c LIMIT 1

```

{ ... }

Summary

One crime node currently under investigation has been returned:

- **Primary Type:** CRIMINAL TRESPASS
- **Date:** 2023-06-15T16:48:00Z
- **Description:** The victim reported observing a BLACK vehicle with license plates beginning with 'EB' leaving the area shortly after the incident occurred.
- **Status:** investigation
- **Location:** POINT(-87.6195518 41.695476192)

extra...

Figure 15.12 Initial query response showing a crime node currently under investigation. The interface shows (a) the Selection panel (populated by double-clicking the crime node) and the Canvas with an information panel displaying details about the current node in the center; and (b) the chat interface with query processing details.

When the crime node is clicked, detailed information about the incident appears in the canvas area, including a narrative that mentions a black vehicle with a partial license plate number starting with "EB". This information will become valuable as our investigation progresses.

In the chat interface, we can see the system's reasoning process, explaining why it chose this crime and how it structured the query to ensure that we received an active investigation. This transparency helps users understand how their natural language questions are being interpreted and executed.

The summary generated by our system highlights key aspects of the crime, including its classification as criminal trespass and the presence of vehicle information in the narrative. This demonstrates how our summary generation component can extract and emphasize relevant details that may be buried in property fields.

This initial interaction demonstrates several capabilities of our system: natural language understanding, appropriate visualization selection, and intelligent summarization of node properties. But more importantly, it sets the stage for increasingly complex queries that will use this context as we proceed with our investigation.

15.3.2 Spatial analysis of surveillance coverage

With our crime node identified, the next logical step is to check for surveillance coverage in the area. Our system's spatial reasoning capabilities allow us to ask about nearby ANPR cameras without needing to specify exact coordinates or distance calculations.

We double-click the crime node to add it to our selection and then enter this prompt: "Return any ANPR camera node located within 1 km from the selected crime." Note how we can refer to "the selected crime" directly; the system understands this context from our current selection. Figure 15.13 captures this interaction, showing how the system responds with both a graph visualization and an interactive map.

The canvas now includes the crime node, along with a newly discovered ANPR camera node in yellow. What makes this response particularly powerful is the system's decision to provide a map visualization alongside the graph view. The map displays two markers—one for the incident location and another for the camera—using colors that match their respective nodes in the graph.

The camera's position appears strategically valuable: it's located near an intersection that could be an entry or exit point for the area where the trespass occurred. This spatial insight, made immediately apparent through the dual visualization, suggests that this camera's data could provide useful leads for our investigation.

This step demonstrates several sophisticated capabilities of our system: spatial query processing, selection-aware natural language understanding, and intelligent visualization selection. The adaptive response, choosing the most appropriate visualization for the data at hand, is a key feature of our expert-emulating approach.

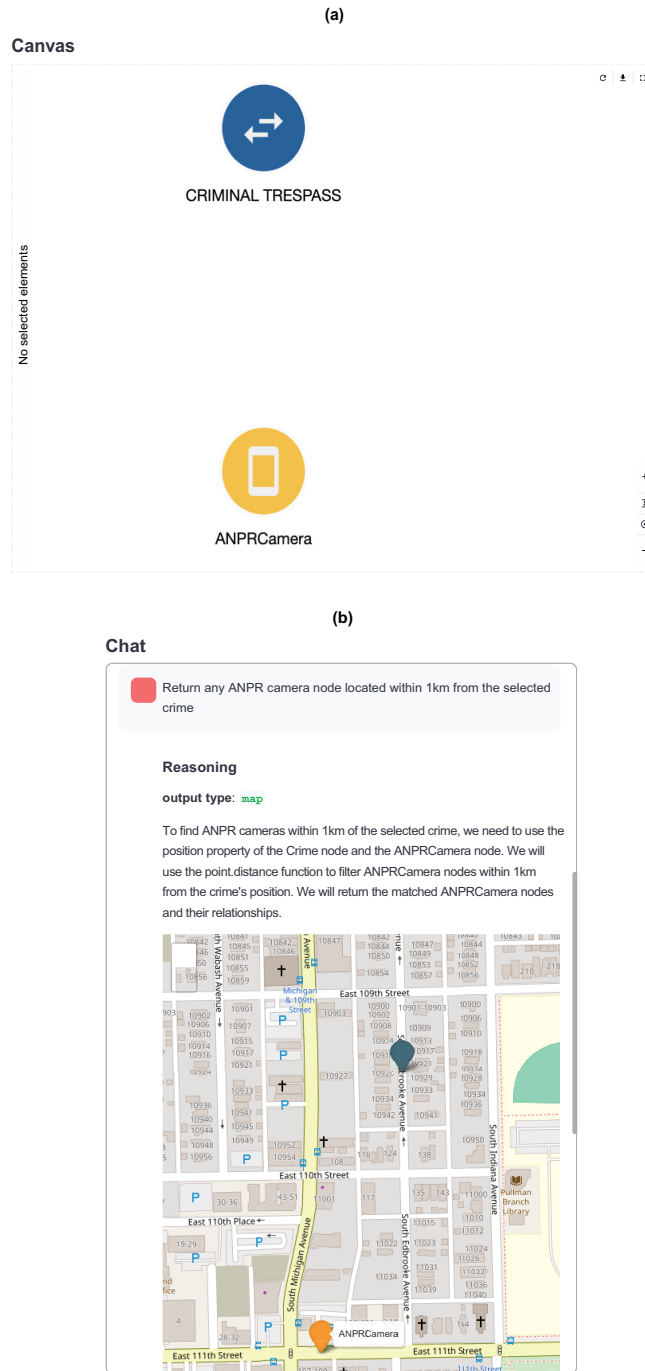


Figure 15.13 Spatial query response showing an ANPR camera near the crime location (a). The system automatically chose a map visualization (b) to display the spatial relationship between the crime and the nearby ANPR camera.

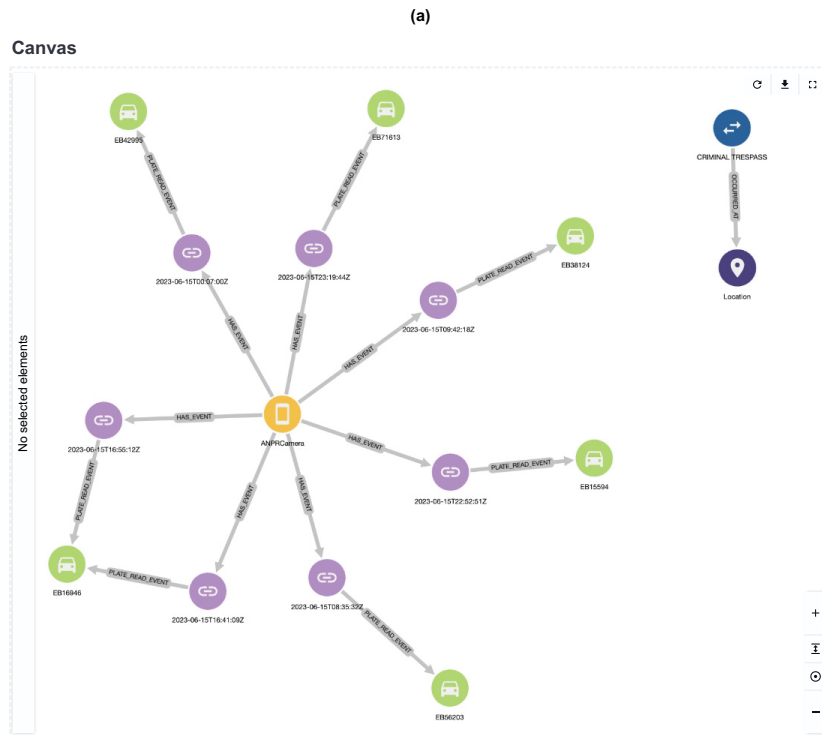
15.3.3 Vehicle pattern detection

We can now search for vehicles that match the description from the incident report. We include the ANPR camera by double-clicking its node, adding it to our current selection alongside the crime node.

Our next prompt uses this selection while adding specific vehicle criteria: “Return the vehicles detected by the selected camera on June 15, 2023. The vehicle is black and its license plate must start with EB.” Note how we can combine references to selected elements (“selected camera”) with explicit constraints about the vehicle’s appearance and license plate. Figure 15.14 shows how the system expands our visualization to include matching vehicles and their detection events.

The graph now displays multiple vehicle nodes connected to our ANPR camera through detection events, with the query’s reasoning visible in the chat interface. The system has understood the temporal constraint (“June 15, 2023”), the vehicle’s physical description (“black”), and the partial plate number (“EB”), incorporating all these elements into a single coherent query.

The summary highlights each matching vehicle, providing an overview of potential suspects. This demonstrates our system’s ability to handle multifaceted queries that combine node selection, temporal constraints, and property matching.



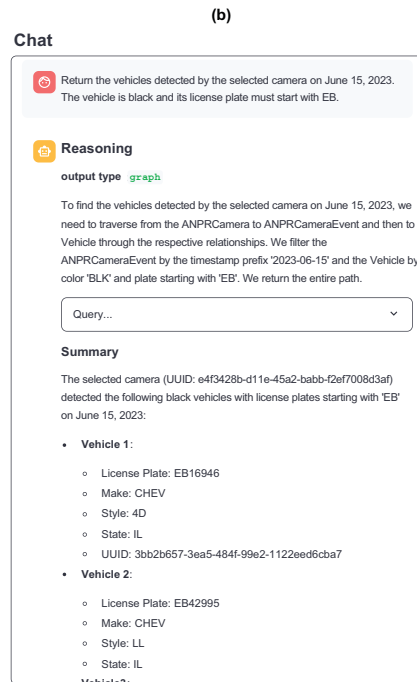


Figure 15.14 Vehicle query results showing matching vehicles and their detection events. Each path represents a complete vehicle detection record, with timestamps visible on the event nodes. The system's response includes both (a) the graph visualization and (b) a detailed summary of each vehicle's properties.

This step marks a significant advancement in our investigation, transforming isolated data points about a crime and a camera location into a set of vehicles that could be connected to our incident. But can we get more insight by providing additional context about our investigative goals? Let's explore this in our next prompt.

15.3.4 Context-aware request refinement

Let's refine our approach by using our expert-emulating system's ability to extract information from selected nodes and perform context-aware summarization. Instead of explicitly stating the vehicle description and date constraints, we can rely on the system to extract these details from our selected crime node. Moreover, by providing context about our role and investigative intent, we can guide the system toward generating more analytical insights.

We rephrase our prompt to reflect this investigative context: "I'm an investigator and I'm working on the selected crime. I need all the vehicle nodes that are compatible with the description and were detected by the selected camera the day of the incident. Are there any that seem significantly more likely to be involved in the incident?" The response, shown in figure 15.15, demonstrates how additional context transforms the system's analysis.



Figure 15.15 Enhanced analysis showing the same vehicle data with investigative context (a). The system augments its response in (b) with an Analysis section that identifies patterns of interest, demonstrating how additional context leads to more insightful summarization of the same underlying data.

Although the query returns the same vehicles as before, the summarization now includes a deeper analysis of the results. The system identifies an interesting pattern: one of the matching vehicles was detected twice around the time of the incident, suggesting a potential circuit of the area that warrants further investigation.

This enhanced response demonstrates how the system can autonomously extract and apply constraints from node properties, eliminating the need for explicit restatement. It also illustrates how providing investigative context—in this case, identifying ourselves as investigators seeking suspicious patterns—enables the system to generate more relevant and insightful summaries.

The system has moved beyond simply matching criteria to actively analyzing temporal patterns that may indicate suspicious behavior. This sets the stage for our final investigative step: examining whether any of these vehicles have connections to known offenders.

15.3.5 *Historical record analysis*

Building on our discovery of a vehicle with suspicious movement patterns, we can further enrich our investigation by considering the criminal history of vehicle owners. This type of background check is a standard investigative practice that our system can integrate naturally into its analysis.

We modify our prompt slightly: “I’m an investigator and I’m working on the selected crime. I need all the vehicle nodes that are compatible with the description and were detected by the selected camera on the day of the incident. Some of these vehicles may be owned by previous offenders. What vehicles are the most likely to be involved in the incident?”

Figure 15.16 reveals a significant breakthrough in our investigation. The system expands its analysis to include ownership relationships and criminal records, discovering that the vehicle previously flagged for suspicious movements is owned by an individual with a relevant criminal history.

Most notably, this person’s record includes a previous conviction for criminal trespass—the same type of offense we’re currently investigating. The summary agent recognizes the heightened significance of this connection, highlighting it in both the general summary and the analysis section.

This final step shows how the system can do all of the following:

- Maintain contextual awareness across multiple query refinements
- Integrate various types of evidence (spatial, temporal, and historical)
- Identify meaningful patterns and connections
- Present findings in a way that directly supports investigative decision-making

Through this investigation, we’ve seen how natural language interaction, intelligent summarization, and context-aware analysis come together to support real-world investigative workflows.

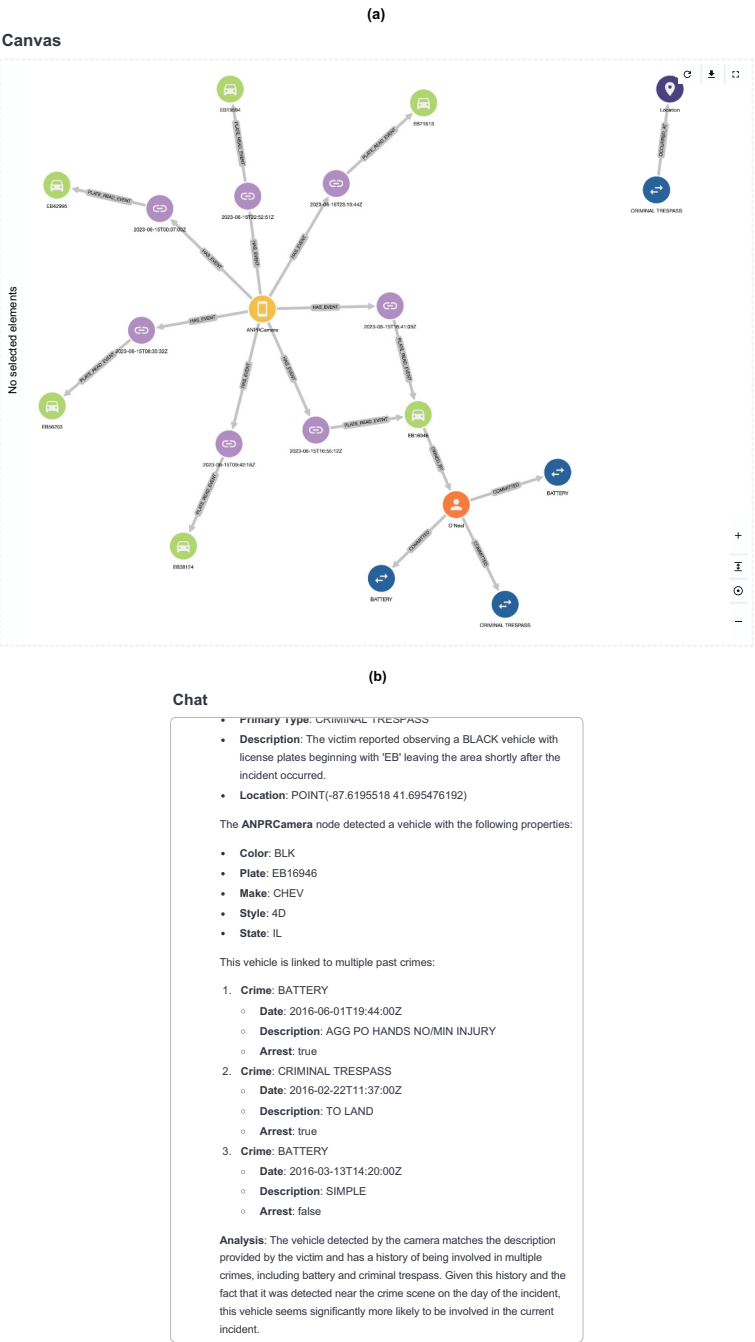


Figure 15.16 Final investigative insight revealing criminal history. (a) The graph expands to show that a vehicle owner has connections to multiple prior crimes, including a previous criminal trespass. (b) The summary provides a detailed breakdown of the prior offenses, demonstrating the system's ability to integrate temporal, spatial, and historical evidence into a cohesive investigative narrative.

15.4 *Future directions and enhancements*

Throughout chapters 14 and 15, we’ve explored the development of a question-answering system for KGs. Rather than treating it as purely a language modeling problem, we’ve built a system that emulates how human experts work with graph databases: understanding schema context and constructing queries through reasoned steps. Our implementation shows how this expert-emulating approach can be realized through a pipeline of specialized agents, each handling distinct aspects of the process.

However, this implementation should be viewed as a foundation rather than as a turnkey solution. Its value lies not just in what it does but also in how it’s built. The foundation’s strength comes from its underlying architecture, particularly in how it approaches observability and expert emulation.

The system’s transparency isn’t merely for debugging; it creates natural points for collecting feedback and identifying improvements. Each component’s decision-making process is visible, making it possible to understand what the system does and also why it does it.

Equally important is the expert-emulation pattern that underlies the design. When faced with new challenges or opportunities for improvement, teams can begin with a simple but powerful question: “What would an expert do in this situation?” This human-centric approach to system design makes improvements feel natural rather than forced; they emerge from understanding and modeling expert behavior.

In the following sections, we’ll explore several paths for building on this foundation. These are examples of how the system’s observable, expert-emulating nature enables specific types of improvements.

15.4.1 *Learning from use*

A natural starting point for system evolution uses one of our core design principles: comprehensive observability. Every query processed by the system generates rich information about user intentions, query patterns, and outcome effectiveness. This observable data creates opportunities for systematic improvement at multiple levels.

The most direct application of this observability lies in collecting and categorizing “complaint-like” questions: prompts in which users indicate that the system’s response didn’t meet their needs. Rather than treating these as failures, our observable pipeline lets us extract meaningful patterns from these instances. By categorizing these patterns using LLM-based analysis, we can surface common user challenges in a dynamic dashboard of pain points that will become a strategic tool for guiding development priorities and resource allocation.

Equally valuable will be the collection of successful interactions. When the system generates queries that users find especially helpful, the chain of reasoning can be preserved. These success patterns can then be systematically analyzed to identify what makes them effective, creating a foundation for enhancing our example database.

By analyzing patterns in user questions, we can also identify clusters of similar queries that may benefit from specialized handling. This understanding can be used to optimize

downstream processes: for example, by selecting only the most relevant portions of the schema or by providing more focused examples for specific types of questions.

This learning from use demonstrates how our foundational architecture enables the system to evolve based on actual use patterns rather than predetermined rules. Improvements emerge naturally while remaining grounded in understandable, expert-like behavior.

15.4.2 Enhancing core capabilities

By consistently asking “What would an expert do?” we can identify and implement improvements that align with human expertise patterns while using our observable pipeline architecture. A key area for enhancement is schema handling, where we can emulate how human experts build their understanding of a KG’s structure. Experts often run preliminary queries to understand how the data is structured and used, going beyond the basic schema definition. We can model this behavior by implementing schema enrichment agents that analyze data patterns and augment the base schema with additional context. This enriched schema information can then flow downstream to improve query generation and result interpretation.

For large-scale KGs, experts typically work with mental models at different levels of abstraction. A multilayer schema management approach could mirror this mental process. For instance, in our investigative KG, the bottom layer would contain all detailed nodes and relationships (crimes, vehicles, camera events, people), and higher layers would organize these into domain-focused views like vehicle monitoring or criminal justice records. The system could use these higher-level views for broad understanding (e.g., which domains to query) while preserving detailed descriptions for specific query generation. This layered approach manages complexity without sacrificing precision, much as human experts zoom in and out of different levels of detail as needed.

15.4.3 Advanced evolution paths

More ambitious evolution paths can significantly enhance the system’s capabilities. These advanced approaches maintain our core principles while incorporating emerging techniques in language model applications.

A particularly promising direction involves moving beyond pure in-context learning for schema understanding and query generation. Our current approach of embedding schema information and examples in prompts is effective, but it faces scalability challenges with larger KGs. Fine-tuning approaches offer an alternative path. By using the schema itself as training data, we can develop KG-aware agents that have a deeper, more efficient understanding of the graph structure and relationships.

Studies have shown that in-context learning, although flexible, consistently underperforms compared to task-specific adaptation approaches [1], [2]. This performance gap persists even when both approaches have access to the same example sets. These findings suggest that moving toward fine-tuned components could significantly improve our system’s performance while potentially reducing its computational overhead.

The transition to fine-tuned components wouldn't require abandoning our expert-emulating architecture. Instead, we could selectively replace or augment in-context learning components with fine-tuned alternatives while maintaining our observable pipeline structure.

This evolution path also opens possibilities for more sophisticated query planning. Fine-tuned agents could develop a more nuanced understanding of query patterns and their relationships with different graph structures, potentially leading to more efficient and effective query generation. The system could maintain its step-by-step reasoning approach while building on a stronger foundation of graph knowledge.

Summary

- An expert-emulating approach mimics how human experts interact with graph databases.
- LangGraph's state-based design enables creation of modular, observable AI pipelines where each component maintains independent reasoning.
- Pipeline integration architecture enables real-time streaming of processing updates to interactive user interfaces.
- Context-aware query generation combines schema knowledge, user selections, and conversational history for natural interactions.
- Multistep analytical workflows can integrate spatial, temporal, and historical analysis in a single coherent process.
- Message history management enables stateful conversations and real-time progress updates.